arXiv:2606.04990v2 [cs.CR] 14 Jun 2026

FROM AGENT TRACES TO TRUST: A SURVEY OF EVIDENCE TRACING AND EXECUTION PROVENANCE IN LLM AGENTS

Yiqi Wang[1,*], Jiaqi Zhang[2], Taotao Cai[3], Zirui Liu[4], Qingqiang Sun[5], Zequn Sun[6], Zhangkai Wu[7], Manqing Dong[7], Mingkai Zheng[8], Xuefei Yin[1], Tianyu Shi[9], Yanming Zhu[1]

[1]Griffith University [2]Jiangsu University [3]University of Southern Queensland
[4]Peking University [5]Great Bay University [6]Nanjing University
[7]The University of Sydney [8]Southern University of Science and Technology [9]McGill University

[*]yiqi.wang.jennie@gmail.com

ABSTRACT

Large language model (LLM)-based agents are rapidly evolving from passive text generators into autonomous systems capable of planning, tool use, retrieval, memory access, environmental interaction, and multi-agent collaboration. While these capabilities expand agent autonomy, they also make agent behavior increasingly difficult to verify, debug, and audit. Final-answer accuracy alone cannot explain how an output was produced, which evidence supported each claim, whether tool calls were justified, how memory influenced later decisions, or where execution failures originated. In response, evidence tracing and execution provenance provide a principled foundation for structuring and reasoning about agent behavior. By modeling the connections among retrieved evidence, tool outputs, memory items, observations, intermediate claims, actions, and final answers, provenance offers a process-level accountability layer for trustworthy LLM agents. In this survey, we treat execution provenance as the full typed graph of an agent execution and evidence tracing as its projection onto evidence-support relations, unifying retrieval grounding, claim support, tool-use safety, memory lineage, observability, debugging, audit, and recovery under a common framework. We introduce a taxonomy covering trace sources, evidence and execution units, provenance relations, tracing granularity and timing, representation forms, and trust functions, and map representative agent systems onto this taxonomy. We then review key methodological directions, including provenance representation, evidence attribution, tool-use provenance, runtime guardrails, provenance-bearing memory, observability, and failure diagnosis. Finally, we connect existing benchmarks, datasets, and metrics to provenance-related capabilities, discuss how evaluation can move beyond final-answer correctness toward process-level accountability, and outline open challenges for unified trace schemas, semantic provenance, provenance-aware safety, realistic execution-trace benchmarks, recovery-oriented evaluation, and privacy-aware audit infrastructure. Related resources are collected and continuously updated in the accompanying GitHub repository.

Keywords LLM Agents, Agent Traces, Evidence Tracing, Execution Provenance, Provenance Graphs, Tool-Use Safety, Memory Provenance, Agent Observability, Runtime Guardrails, Trustworthy AI

1 Introduction

Large language model (LLM)-based agents are emerging as a powerful paradigm for complex task execution, moving beyond standalone text generation toward interactive execution pipelines that support reasoning, tool use, retrieval, memory access, environment interaction, and multi-agent collaboration Schick et al. [2023], Yao et al. [2023], Shinn et al. [2023]. As these capabilities expand, recent systems and benchmarks show that LLM agents increasingly operate across tools, external knowledge sources, code execution, web or GUI environments, and multi-agent workflows, creating complex execution contexts in which many intermediate sources, actions, and observations may jointly shape the final output Qin et al. [2024], Liu et al. [2024], Zhou et al. [2024], Yao et al. [2025].

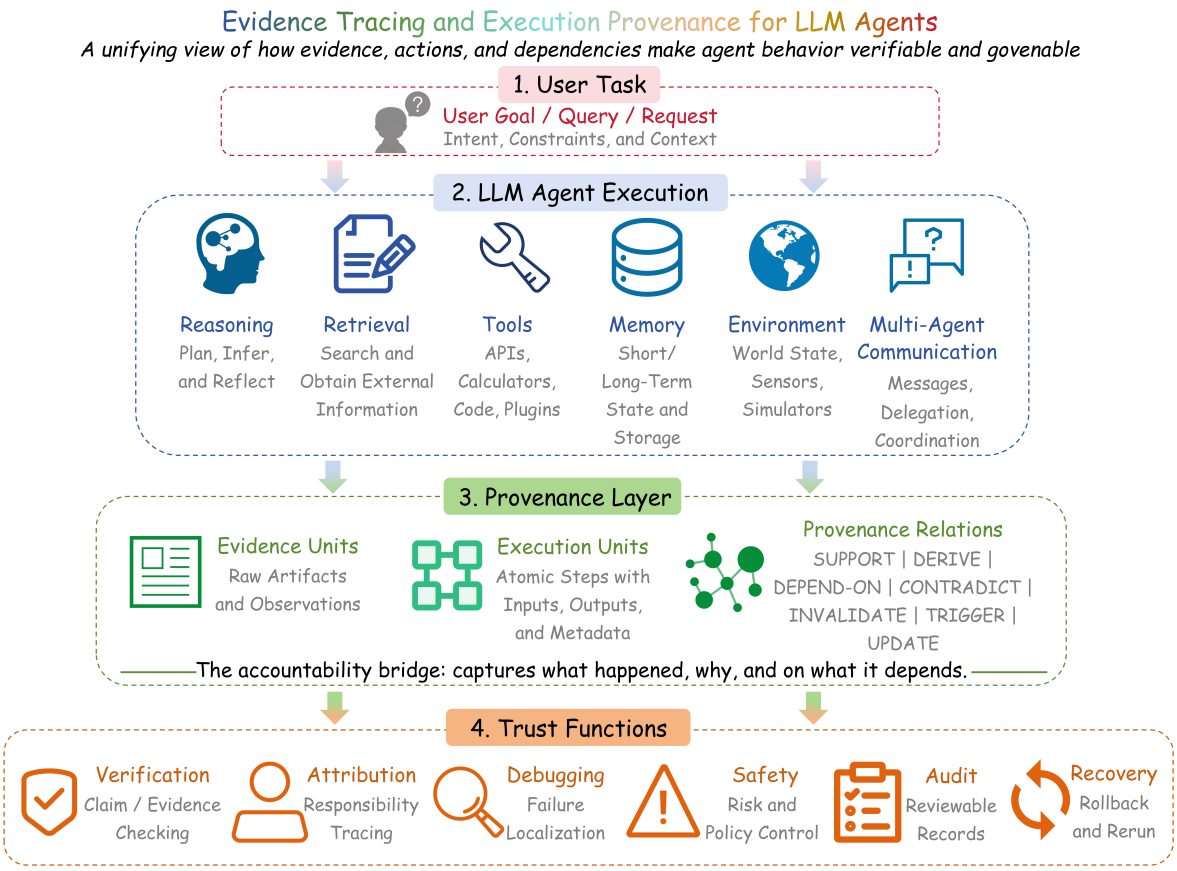


Figure 1: Overview of the proposed provenance perspective. Agent executions produce heterogeneous units that are connected through typed relations and used for verification, attribution, debugging, safety, audit, and recovery.

This creates a process-level accountability gap in LLM agent systems. Final-answer accuracy evaluates only the endpoint of an execution; it does not explain how an output was produced, which evidence supported each claim, whether tool calls were justified, whether memory items were relevant and trustworthy, or which execution step caused a failure. Recent studies make this limitation concrete: tool-safety benchmarks show that risks can arise from intermediate tool-use decisions, adversarial tool interactions, realistic tool environments, and MCP-based workflows Ruan et al. [2024], Debenedetti et al. [2024], Vijayvargiya et al. [2026], Zong et al. [2025], while trace-oriented and multi-agent failure studies show that errors often require localizing responsible steps, components, agents, or error modes within long executions Deshpande et al. [2025], Cemri et al. [2026], Zhang et al. [2025a], Kong et al. [2025]. These findings suggest that trustworthy agent systems require mechanisms that can record, connect, and reason over the evidence and execution steps underlying agent behavior.

In this survey, we examine this need through the lens of *evidence tracing* and *execution provenance*. Evidence tracing focuses on identifying and connecting evidence units that support, contradict, invalidate, or influence agent claims, decisions, and actions. Execution provenance provides a broader structured account of how an agent execution unfolds, covering retrieved documents, tool calls, parameters, observations, memory accesses, intermediate claims, actions, inter-agent messages, and final outputs. This perspective is informed by provenance and trace modeling in software and distributed systems, including W3C PROV-DM for representing entities, activities, agents, and provenance relations World Wide Web Consortium [2013], OpenTelemetry for distributed execution traces OpenTelemetry [2026], and recent agent observability systems such as AgentOps and AgentTrace Dong et al. [2024], AISayyad et al. [2026].

Terminology. We use *execution provenance* to denote the complete typed representation of an agent run, including evidence units, execution units, and their causal, procedural, dependency, update, contradiction, and invalidation relations. We use *evidence tracing* to denote the projection of this provenance structure onto evidence-support and influence relations between evidence units and agent claims, decisions, or actions. Thus, evidence tracing is a sub-problem of execution provenance: every evidence-support link belongs to the provenance structure, while

execution provenance additionally records how tools, memory, observations, actions, and inter-agent messages shape the end-to-end execution.

Several research threads provide partial but fragmented foundations for *evidence tracing* and *execution provenance*, including evidence grounding, tool-use safety, agent memory, graph-based methods, and observability. However, these threads usually capture only one slice of the provenance problem—evidence support, action influence, memory lineage, relational structure, or runtime inspection—rather than a unified account of how evidence, tools, memory, observations, claims, actions, and agents jointly shape execution outcomes. This survey connects these slices under a process-level provenance view.

Figure 1 summarizes this perspective. User tasks initiate heterogeneous agent executions across reasoning, retrieval, tool use, memory, environments, and multi-agent communication. The provenance layer connects evidence units, execution units, and typed provenance relations, linking agent execution to downstream trust functions such as verification, attribution, debugging, safety, audit, and recovery.

Survey scope and analytical lens. This survey covers works that use provenance-bearing information to trace, represent, evaluate, or govern LLM-agent executions, including evidence, tools, memory, observations, claims, actions, inter-agent messages, and final outputs. We discuss RAG, memory, graph-based retrieval, tool-use safety, and observability only when they support evidence tracing, execution dependency modeling, or process-level accountability. We organize the survey by provenance units, relations, granularity, timing, representations, trust functions, and evaluation, using four design tensions as the analytical lens: logging versus semantic provenance, component tracing versus end-to-end accountability, static schemas versus runtime enforcement, and completeness versus privacy and scalability.

In summary, this survey makes the following contributions:

- We formulate evidence tracing and execution provenance as a process-level accountability perspective for trustworthy LLM agents.
- We introduce a taxonomy covering trace sources, evidence and execution units, provenance relations, tracing granularity and timing, representation forms, and trust functions.
- We review key methodological directions, including evidence attribution, tool-use provenance, runtime safety control, memory provenance, observability, and failure diagnosis.
- We summarize benchmarks, datasets, and metrics for evaluating provenance-related capabilities beyond final-answer correctness.
- We identify open challenges for provenance-aware LLM agents, including unified trace schemas, semantic provenance, realistic execution-trace benchmarks, recovery-oriented evaluation, and privacy-aware audit infrastructure.

Structure of the Survey. The survey follows a single through-line: turning raw agent traces into accountable provenance. Section 2 introduces the necessary background on LLM agents, agent traces, and process-level accountability, while Section 3 presents our taxonomy of trace sources, evidence and execution units, provenance relations, tracing granularity and timing, representation forms, and trust functions. Section 4 reviews provenance representations, ranging from structured logs to execution graphs, evidence graphs, claim-support graphs, and runtime provenance structures. Sections 5 and 6 examine two core settings where provenance failures are especially consequential: tool use, where agent actions affect external systems, and memory, where evidence persists and influences future executions. Finally, Section 7 maps benchmarks and metrics to provenance-related capabilities, Section 8 discusses open challenges and future directions, and Section 9 concludes the survey.

2 Preliminaries: Agent Traces and Provenance

This section establishes the basic concepts used throughout the survey: agent executions, agent traces, process-level accountability, and provenance.

2.1 LLM Agent Executions

LLM agents extend language models from single-turn response generation to goal-directed execution. During an execution, an agent may retrieve evidence, call tools, observe environments, read or update memory, reason over intermediate states, and communicate with other agents. Thus, an agent output is the result of a heterogeneous execution trajectory rather than a standalone final response.

2.2 Agent Traces

We use *agent trace* to denote the recorded artifacts generated or consumed during an agent run, including instructions, reasoning steps, retrieval queries, retrieved documents, tool calls, tool outputs, memory operations, observations, intermediate claims, inter-agent messages, actions, and final responses. Such traces expose the intermediate evidence, actions, memory, and observations that shape final behavior.

2.3 From Accuracy to Process-Level Accountability

Final-answer accuracy is insufficient for evaluating LLM agents because two executions with the same answer may differ in reliability, safety, and auditability. A correct answer may rely on unsafe tool use, irrelevant evidence, stale memory, or policy-violating actions, while an incorrect answer may originate from misleading evidence, malformed parameters, environment misobservation, or multi-agent miscoordination. Process-level accountability therefore asks not only whether the final output is correct, but also how the agent reached it.

2.4 Traceability and Provenance

We use *traceability* to denote the ability to record and inspect an agent's execution path, and *provenance* to denote the structured relationships among trace elements. In short, traces record what happened, while provenance explains how execution artifacts are connected, such as which evidence supports a claim, which observation triggers an action, or which tool output updates memory. This distinction motivates the taxonomy in the next section.

3 Taxonomy of Evidence Tracing and Execution Provenance

This section develops a taxonomy that identifies the main dimensions along which provenance-aware agent systems can be described and compared, rather than cataloguing individual systems. We organize it around five dimensions: trace-generating sources, recordable evidence and execution units, provenance relations, recording policy, and trust functions, as summarized in Figure 2.

The taxonomy is motivated by two complementary traditions. In provenance modeling, W3C PROV-DM represents provenance through entities, activities, and agents, together with relations such as usage, generation, and derivation World Wide Web Consortium [2013]. This provides a foundation for modeling how data objects or claims are produced. In distributed observability, OpenTelemetry models executions as traces composed of spans and causal relationships, providing a useful analogy for representing multi-step execution paths OpenTelemetry [2026]. However, LLM agents introduce additional semantic and procedural artifacts that are not fully captured by traditional provenance or observability alone, including retrieved evidence, generated claims, tool-call rationales, memory items, natural-language observations, and inter-agent messages. Agent-oriented observability frameworks such as AgentOps and AgentTrace further show that LLM-agent systems require structured records of both execution artifacts and associated context Dong et al. [2024], AlSayyad et al. [2026]. Building on these foundations, we define a taxonomy tailored to the provenance structure of LLM agent execution.

3.1 Trace Sources

Trace sources refer to the components of an agent system that generate provenance-relevant artifacts. In simple language-generation settings, the main trace source may be the model output itself. In LLM agents, however, traces can originate from reasoning, retrieval, tool use, memory, environment interaction, and multi-agent communication. Each source contributes different types of evidence and execution records, and each introduces different failure modes.

Reasoning traces include intermediate thoughts, plans, decompositions, reflections, critiques, and self-corrections generated by the agent. Systems such as ReAct and Reflexion show that reasoning traces can influence action selection and future behavior Yao et al. [2023], Shinn et al. [2023]. From a provenance perspective, reasoning traces are important because they may explain why an action was selected, why a document was considered relevant, or how an intermediate claim contributed to the final answer.

Retrieval traces include search queries, retrieved documents, passages, snippets, citation candidates, ranking scores, and retrieval-time metadata. Work on citation generation, factual precision, and RAG evaluation shows that generated answers should be evaluated not only by final correctness, but also by whether their claims are supported by appropriate evidence Gao et al. [2023], Min et al. [2023], Es et al. [2024], Saad-Falcon et al. [2024], Ru et al. [2024], Niu et al. [2024]. Retrieval traces therefore provide the basis for claim-level support, contradiction detection, and evidence attribution.

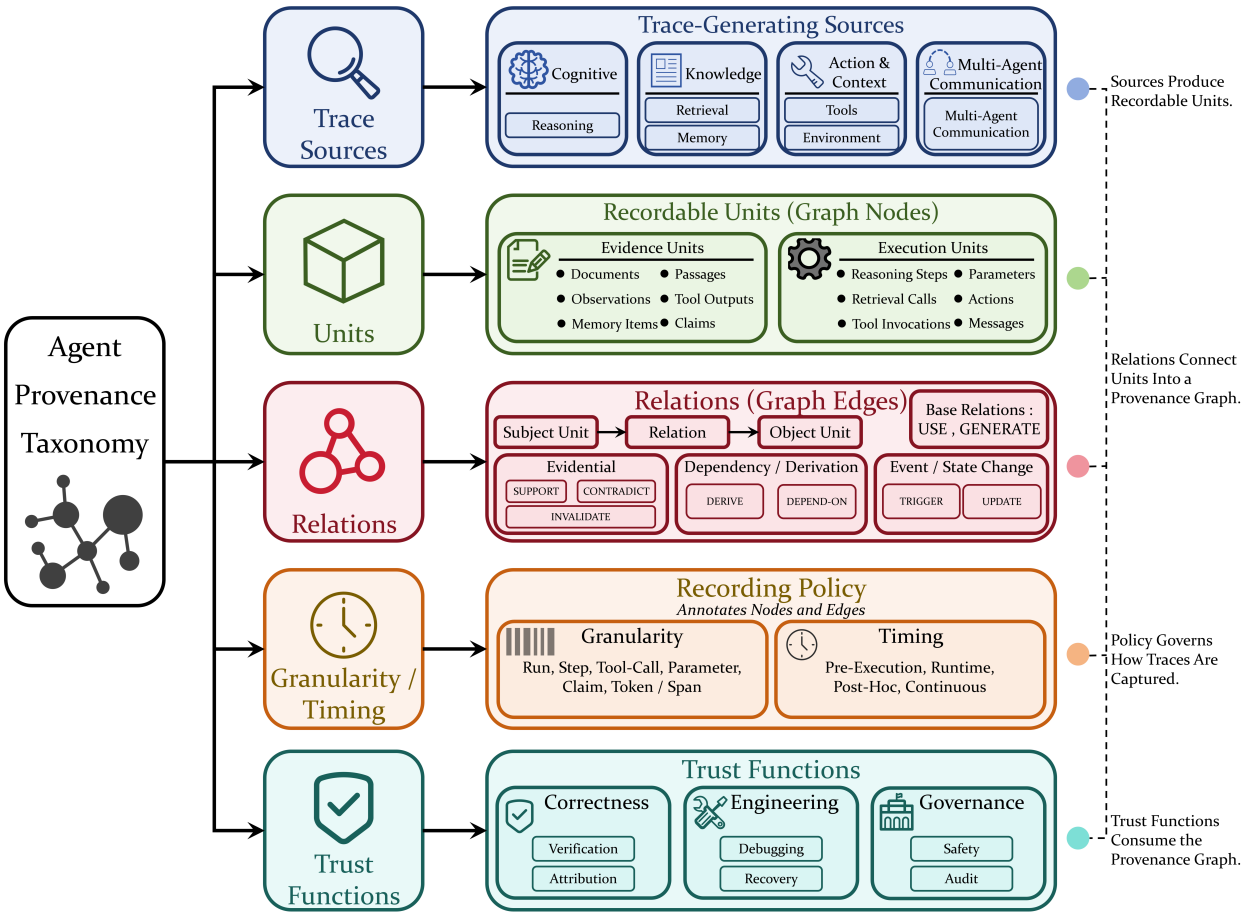


Figure 2: Taxonomy overview of agent provenance in this survey. The taxonomy organizes LLM-agent provenance along five dimensions: trace sources, evidence and execution units, provenance relations, granularity and timing, and trust functions.

Tool traces record tool names, API calls, arguments, execution results, errors, permissions, execution boundaries, and downstream state changes. Tool-use systems and benchmarks such as Toolformer, ToolLLM, WebArena, τ-bench, MCP-SafetyBench, and AgentBound show that tool calls are central to agent execution and may span APIs, external environments, MCP servers, host mediation layers, and multi-server workflows Schick et al. [2023], Qin et al. [2024], Zhou et al. [2024], Yao et al. [2025], Zong et al. [2025], Bühler et al. [2025]. Because tool calls can affect external systems, their provenance must capture what was invoked, why it was invoked, where its arguments came from, which permissions or boundaries governed the call, and how its output influenced later claims or actions.

Memory traces record memory reads, writes, updates, deletions, consolidation steps, and retrieved memory items. Systems such as Generative Agents, MemGPT, MemoryBank, and A-MEM demonstrate that long-term memory can influence agent behavior across sessions or tasks Park et al. [2023], Packer et al. [2023], Zhong et al. [2024], Xu et al. [2026]. Memory traces are provenance-relevant because a current output may depend on information introduced in a previous episode, summarized by the agent, or updated over time. Without memory provenance, it is difficult to determine whether a decision was grounded in current evidence or stale, biased, or incorrectly summarized memory.

Environment traces capture observations and actions in external environments, such as web pages, graphical interfaces, code repositories, databases, simulators, or embodied worlds. WebArena and τ-bench demonstrate that agent success often depends on multi-step interaction with changing environment states, policy constraints, and final external database states Zhou et al. [2024], Yao et al. [2025]. In such settings, the provenance of an output includes not only text evidence but also the sequence of observed states and executed actions that changed the environment.

Multi-agent communication traces record messages, roles, delegations, critiques, task decompositions, and intermediate outputs exchanged among agents. AutoGen and CAMEL show that multi-agent workflows distribute reasoning and

action across agents Wu et al. [2024a], Li et al. [2023]. Provenance must therefore capture how claims, decisions, and failures propagate across agents, including responsible agents, failure-inducing steps, error modes, and counterfactual or contrastive trajectories for attribution Zhang et al. [2025a], Kong et al. [2025].

3.2 Evidence and Execution Units

Trace sources produce different kinds of units. We distinguish between *evidence units*, which provide semantic support or context for claims and decisions, and *execution units*, which record procedural steps in agent behavior. The distinction is analytical rather than strict: a tool output, for example, may be both an execution result and an evidence unit supporting a later claim.

Evidence units include documents, passages, tables, retrieved snippets, citations, environment observations, memory items, tool outputs, user instructions, policies, and intermediate claims. Prior work on citation-based generation and factual evaluation provides useful foundations for defining evidence units. ALCE studies citation-supported generation, FActScore decomposes long-form generation into atomic factual claims, FEVER formulates verification around claims and evidence sentences, and RAGChecker provides fine-grained diagnosis of retrieval and generation behavior Gao et al. [2023], Min et al. [2023], Thorne et al. [2018], Ru et al. [2024]. These works suggest that provenance should not only link an answer to a document, but also connect specific claims to specific supporting or contradicting evidence.

Execution units include reasoning steps, retrieval calls, tool invocations, parameter-generation steps, memory operations, environment actions, inter-agent messages, and final outputs. Observability-oriented systems such as AgentOps and AgentTrace emphasize the need to record agent artifacts and execution-level data for inspection and debugging Dong et al. [2024], AlSayyad et al. [2026]. Compared with evidence units, execution units are often temporal and procedural: they indicate what happened during the run, in what order, and under what context.

For LLM agents, a useful provenance representation should cover both types. If only evidence units are recorded, the system may know which document supports a claim but not which tool call produced the evidence or which action used it. If only execution units are recorded, the system may know that a tool was called but not whether its output actually supported the final answer. Evidence tracing and execution provenance therefore need to connect semantic support with procedural execution.

3.3 Provenance Relations

Provenance relations define how trace units are connected. In traditional provenance models, relations such as *used*, *wasGeneratedBy*, and *wasDerivedFrom* describe how entities, activities, and agents participate in the production of an artifact World Wide Web Consortium [2013]. These relations provide a useful foundation, but LLM agents require additional relation types because their executions combine semantic evidence use, tool-mediated actions, memory updates, environment observations, and multi-agent communication. In this setting, provenance must capture not only which artifact was used or generated, but also whether a piece of evidence supports a claim, whether a tool output contradicts memory, whether an observation invalidates a plan, or whether an error triggers a recovery step.

Building on traditional provenance relations, we define seven relation types for LLM-agent provenance: SUPPORT, DERIVE, DEPEND-ON, CONTRADICT, INVALIDATE, TRIGGER, and UPDATE. These relations capture two complementary aspects of agent execution: semantic evidence relationships, such as support and contradiction, and procedural execution relationships, such as dependency, triggering, and state update. We additionally retain USE and GENERATE as PROV-compatible base relations.

Several of these relations are motivated by evidence-based generation and RAG evaluation. SUPPORT links evidence units to claims, actions, or answers that they justify. For example, a retrieved passage may support an intermediate claim, or a tool output may support a final answer. This relation is central to citation generation, factual precision, source checking, and faithfulness evaluation Gao et al. [2023], Min et al. [2023], Es et al. [2024], Saad-Falcon et al. [2024], Ru et al. [2024], Wu et al. [2024b]. CONTRADICT links units that conflict semantically, such as a memory item that conflicts with a newly retrieved document. This relation is closely related to hallucination and factual inconsistency analysis, where generated claims may be unsupported or contradicted by evidence Niu et al. [2024], Lin et al. [2025]. INVALIDATE captures a stronger form of negative evidence, where a new observation or tool result makes a prior assumption, plan, or claim no longer usable. For example, a failed API response may invalidate the planned next action.

Other relations capture procedural and state-changing aspects of agent execution. DERIVE captures transformation or summarization, such as deriving a memory item from a previous conversation or deriving a claim from a table. This relation is important for memory-augmented agents, where later behavior may depend on compressed or updated representations of past interactions Park et al. [2023], Packer et al. [2023], Zhong et al. [2024], Xu et al. [2026]. DEPEND-ON represents execution dependency, such as an action depending on a tool result or a tool call depending on

Table 1: Taxonomy of evidence tracing and execution provenance in LLM agents. The taxonomy organizes agent provenance along trace sources, evidence and execution units, provenance relations, tracing design dimensions, and trust functions.

Dimension	Categories	Description	Representative Work
Trace sources	Reasoning, retrieval, tool use, MCP server/host boundaries, memory, environment, multi-agent communication	Components and execution boundaries that generate provenance-relevant artifacts during agent execution.	ReAct, Reflexion, ToolLLM, WebArena, AutoGen, AgentTrace, MCP-SafetyBench Yao et al. [2023], Shinn et al. [2023], Qin et al. [2024], Zhou et al. [2024], Wu et al. [2024a], AlSayyad et al. [2026], Zong et al. [2025]
Evidence units	Documents, passages, observations, tool outputs, memory items, claims, policies, final answers	Semantic objects that support, contradict, invalidate, or contextualize agent claims, actions, and outputs.	ALCE, FActScore, FEVER, RAGChecker Gao et al. [2023], Min et al. [2023], Thorne et al. [2018], Ru et al. [2024]
Execution units	Reasoning steps, retrieval calls, tool invocations, parameters, tool manifests, permission scopes, memory operations, actions, inter-agent messages	Procedural objects that record what the agent did, in what order, under which context, and across which execution boundary.	AgentOps, AgentTrace, TRAIL, AgenTracer, Aegis Dong et al. [2024], AlSayyad et al. [2026], Deshpande et al. [2025], Zhang et al. [2025a], Kong et al. [2025]
Provenance relations	Support, derive, depend-on, contradict, invalidate, trigger, update, use, generate	Typed links that connect evidence and execution units into provenance structures.	W3C PROV-DM, AgentSentry World Wide Web Consortium [2013], Sequeira et al. [2026]
Tracing granularity	Run-level, step-level, tool-call-level, parameter-level, claim-level, token/span-level	Determines how fine-grained the recorded provenance structure is.	FActScore, SourceCheckup, Agent-Sentry Min et al. [2023], Wu et al. [2024b], Sequeira et al. [2026]
Tracing timing	Pre-execution, runtime, post-hoc, continuous	Determines when trace collection, verification, enforcement, or diagnosis occurs.	AgentSpec, Agent-Sentry, TRAIL Wang et al. [2025a], Sequeira et al. [2026], Deshpande et al. [2025]
Trust functions	Verification, attribution, debugging, safety enforcement, audit, failure attribution, recovery	Downstream purposes served by evidence tracing and execution provenance.	RAGAS, AgentOps, AgentTrace, LADYBUG, AgenTracer, OpenAgentSafety Es et al. [2024], Dong et al. [2024], AlSayyad et al. [2026], Rorseth et al. [2025], Zhang et al. [2025a], Vijayvargiya et al. [2026]

generated parameters. This relation is especially important for tool-use safety, because unsafe actions can arise when tool arguments depend on untrusted or injected content Debenedetti et al. [2024, 2025], Costa et al. [2025], Cai et al. [2026], Sequeira et al. [2026]. TRIGGER represents causal or procedural activation, such as an observation triggering a diagnostic tool call or an error triggering replanning. UPDATE captures state changes, including memory updates, environment updates, or database modifications, which are central in interactive benchmarks and tool-agent-user workflows Zhou et al. [2024], Yao et al. [2025].

These relations are useful because they turn raw traces into structured provenance. A chronological log may show that a document was retrieved, a tool was called, and an answer was produced. A provenance graph can additionally show that the document supported one claim, the tool output contradicted another, the contradiction invalidated a previous assumption, and the invalidation triggered a recovery step. Observability and debugging systems similarly show that structured traces are more useful than flat logs for inspection, issue localization, and recovery Dong et al. [2024], AlSayyad et al. [2026], Deshpande et al. [2025], Rorseth et al. [2025]. This relational structure is what allows provenance to support verification, attribution, debugging, safety enforcement, audit, and recovery.

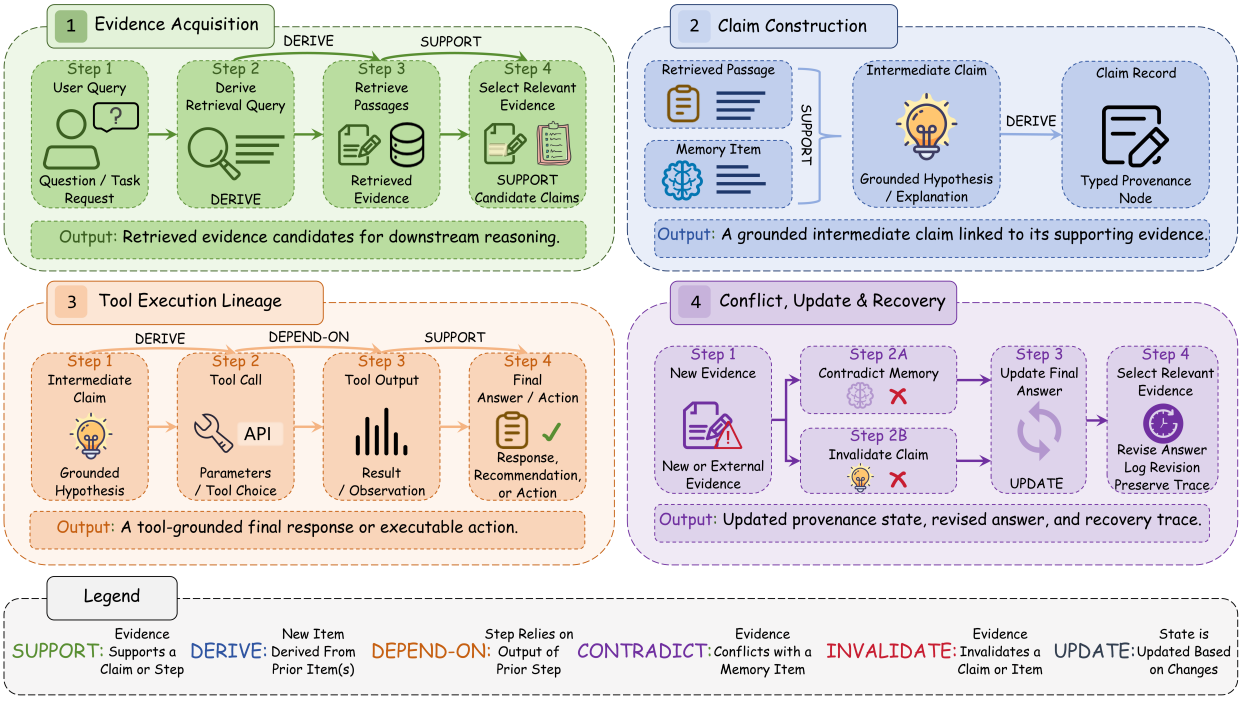


Figure 3: Example provenance graph for an LLM-agent execution. The graph illustrates how user queries, retrieved passages, intermediate claims, tool calls, tool outputs, memory items, new evidence, and final actions can be connected through typed provenance relations such as SUPPORT, DERIVE, DEPEND-ON, CONTRADICT, INVALIDATE, and UPDATE.

Figure 3 illustrates how these relations can be instantiated in a concrete LLM-agent execution. Rather than treating the trace as a flat chronological log, the example connects evidence, claims, tool calls, tool outputs, memory items, new evidence, and final actions through typed provenance relations.

Table 2 makes the relationship to the W3C PROV data model explicit. The upper block aligns agent provenance with PROV entities, activities, agents, and core relations, showing that much of agent execution can reuse a standard provenance vocabulary. The lower block lists what PROV cannot express: agent execution introduces *semantic* relations such as SUPPORT and CONTRADICT, which depend on comparing the content of evidence and claims rather than on bookkeeping about how artifacts were produced. This is the precise sense in which agent provenance extends, rather than merely instantiates, classical provenance models.

3.4 Tracing Granularity and Timing

Tracing design depends on both granularity and timing. Granularity determines how detailed the recorded provenance is, while timing determines when traces are collected, checked, or used.

Run-level tracing records the overall input, major steps, and final output of an agent run. It is useful for coarse audit and reproducibility, but often insufficient for claim-level verification or failure localization. **Step-level tracing** records each reasoning step, retrieval step, tool call, memory operation, or environment action. This level is useful for debugging and process inspection. **Tool-call-level and parameter-level tracing** records the exact tool name, arguments, sources of argument values, tool outputs, errors, and downstream actions. This is important for safety because tool misuse often arises from unsafe or untrusted parameter values. Agent-Sentry, for example, uses execution provenance to track the sources of values that flow into tool arguments Sequeira et al. [2026]. **Claim-level tracing** links individual generated claims to supporting, contradicting, or missing evidence. Work such as FActScore and SourceCheckup shows why document-level citation is often too coarse for factual attribution Min et al. [2023], Wu et al. [2024b]. **Token- or span-level tracing** is the finest level, but it is expensive and may not always be semantically meaningful for agent-level accountability.

Timing introduces another design axis. **Pre-execution tracing** analyzes intended actions before they occur, such as checking whether a tool call is authorized or whether its arguments come from trusted sources. **Runtime tracing**

Table 2: Explicit correspondence between the W3C PROV data model [World Wide Web Consortium, 2013] and the agent-provenance model used in this survey. The upper block maps PROV constructs to agent counterparts; the lower block lists agent-specific semantic relations and units that PROV does not express. This turns the recurring PROV/OpenTelemetry analogy into a concrete mapping.

W3C PROV construct	**Agent-provenance counterpart**	**Notes and agent-specific extension**
PROV-aligned core (reusable vocabulary)		
Entity	Evidence / execution artifact	Documents, passages, tool outputs, memory items, generated claims, messages.
Activity	Execution unit	Reasoning steps, retrieval and tool calls, memory operations, environment actions.
Agent	Agent	LLM, tool/API, user, or sub-agent; multi-agent roles and delegation become first-class.
`used`	DEPEND-ON, USE	An action or tool call depends on a parameter, tool output, or memory item; agent provenance adds trust/taint labels on the used value.
`wasGeneratedBy`	GENERATE	A tool output is generated by a tool call; a claim by a reasoning step.
`wasDerivedFrom`	DERIVE	Memory derived from a document, or a claim from a table; captures summarization and transformation, not only copying.
`wasInformedBy`	TRIGGER	An observation triggers a tool call; an error triggers replanning.
`wasInvalidatedBy`	INVALIDATE	PROV invalidation ends an entity's existence; agent INVALIDATE marks a claim, plan, or memory item as epistemically invalid while the record persists.
`wasRevisionOf`	UPDATE	Memory or external-state updates across turns or sessions.
Agent-only (no PROV counterpart)		
—	SUPPORT	Evidence supports a claim, decision, or action; central to attribution, but PROV has no support/justification semantics.
—	CONTRADICT	A tool output contradicts a memory item or retrieved passage; requires semantic comparison, not provenance bookkeeping.
—	Semantic units	Generated claims, tool-call rationales, natural-language observations, inter-agent messages, and trust labels, which PROV treats as opaque entities.

records and checks provenance during execution, enabling policy enforcement, information-flow control, and early intervention Costa et al. [2025], Cai et al. [2026], Wang et al. [2025a], Sequeira et al. [2026]. **Post-hoc tracing** reconstructs provenance after execution for debugging, audit, and evaluation. Debugging systems such as TRAIL and LADYBUG illustrate the value of trace-based failure localization after an agent run Deshpande et al. [2025], Rorseth et al. [2025]. **Continuous tracing** maintains provenance across sessions, memory updates, and long-horizon workflows, which is especially important for persistent agents and multi-agent systems.

Different applications require different trade-offs. A low-risk question-answering agent may only need claim-level citation and post-hoc audit. A tool-using financial, medical, or enterprise agent may require runtime provenance checking at the tool-call and parameter level. A long-term personalized agent may require continuous memory provenance to track how past interactions influence future behavior. Therefore, tracing granularity and timing should be selected based on the risk, persistence, and external impact of the agent workflow.

3.5 Trust Functions

The purpose of tracing is not merely to store more logs. Traceability and provenance support specific trust functions that improve the reliability, safety, and governability of LLM agents. We summarize six major functions.

Verification checks whether claims, actions, or outputs are supported by appropriate evidence. In RAG and evidence-based generation, verification asks whether generated claims are faithful to retrieved context or cited sources Gao et al. [2023], Es et al. [2024], Saad-Falcon et al. [2024], Ru et al. [2024]. In agent execution, verification also asks whether tool calls, memory use, and environment actions are justified by the available context.

Attribution identifies which evidence units, tool outputs, memory items, or agent messages contributed to a claim, action, or final answer. Attribution is narrower than full provenance but essential for accountability. Claim-level

attribution is especially important because document-level citation may hide unsupported or partially supported claims Min et al. [2023], Wu et al. [2024b], Schreieder et al. [2025].

Debugging uses traces to localize failures. A failed agent run may result from wrong retrieval, malformed parameters, tool execution errors, stale memory, invalid reasoning, or inter-agent miscoordination. Trace-based debugging systems such as TRAIL and LADYBUG show how execution traces can support issue localization and targeted repair Deshpande et al. [2025], Rorseth et al. [2025]. This function turns provenance from a passive record into a diagnostic tool.

Safety enforcement uses provenance to prevent unsafe actions before they occur. Tool-integrated agents may be exposed to indirect prompt injection, untrusted external content, and unsafe tool outputs Ruan et al. [2024], Zhan et al. [2024], Debenedetti et al. [2024]. Information-flow and runtime enforcement methods such as CaMeL, FIDES, NeuroTaint, AgentSpec, and Agent-Sentry show that safe execution requires tracking the origin, transformation, and influence of information throughout the agent workflow Debenedetti et al. [2025], Costa et al. [2025], Cai et al. [2026], Wang et al. [2025a], Sequeira et al. [2026].

Audit reconstructs agent executions for inspection, compliance review, and responsibility analysis. Unlike debugging, which localizes failures, audit asks whether execution records are complete, interpretable, durable, and suitable for external review. This view builds on provenance models such as W3C PROV, which represents entities, activities, agents, and their relations [World Wide Web Consortium, 2013]. For LLM systems, audit trails define chronological records for accountability and post-hoc inspection [Ojewale et al., 2026]. For agent systems, observability and auditability frameworks capture cognitive, operational, and contextual traces across executions [Dong et al., 2024, AlSayyad et al., 2026, Nian et al., 2026].

Recovery uses provenance to repair failed executions and control unsafe downstream effects. Self-reflective and tool-interactive methods support retry-based repair using feedback from failed attempts or external tools [Shinn et al., 2023, Gou et al., 2024]. Failure-analysis work highlights trajectory-level localization as a prerequisite for repair [Zhu et al., 2025]. Transactional planning and semantic recoverability mechanisms support compensation, rollback, and safe reuse of states or tool effects [Chang and Geng, 2025, Yang et al., 2026a]. Checkpoint-restore work shows that rollback can itself create semantically unsafe re-executions [Zheng et al., 2026].

3.6 Mapping Representative Systems onto the Taxonomy

Having defined the five dimensions, Table 3 places representative agent systems within them, so that the taxonomy can be used as a comparison framework rather than only a vocabulary. Three patterns stand out. First, *trace-source* systems such as ReAct, MemGPT, and AutoGen produce rich execution units but implement no trust function on their own; they are substrates that provenance mechanisms build upon. Second, *attribution* systems such as ALCE, FActScore, and SourceCheckup operate post-hoc at claim granularity over retrieval, and rarely touch tools, memory, or actions. Third, *safety* systems such as CaMeL, FIDES, NeuroTaint, and Agent-Sentry operate at runtime on tool parameters, but provide little claim-level evidence support. No single system spans trace sources, fine granularity, runtime timing, an explicit representation, and multiple trust functions at once—which is precisely the gap that a unified provenance layer must close.

3.7 Core Design Tensions in Agent Provenance

The taxonomy above exposes several recurring design tensions in provenance-aware LLM agents. First, there is a tension between *chronological logging* and *semantic provenance*. Logs can record what happened during an execution, but they do not by themselves explain whether a retrieved passage supported a claim, whether a tool call was justified, or whether a memory item influenced a downstream decision. Second, there is a tension between *component-level tracing* and *end-to-end accountability*. Existing methods often focus on retrieval, tool use, memory, observability, or debugging in isolation, whereas agent failures may propagate across these components. Third, there is a tension between *static provenance schemas* and *runtime enforcement*. Static schemas provide reusable object and relation types, but safety, intervention, and recovery often require online source tracking, policy checking, and dependency-aware execution control. Fourth, there is a tension between *trace completeness* and *deployability*. Fine-grained provenance can support verification, debugging, audit, and recovery, but it also increases storage overhead, privacy exposure, annotation cost, and system complexity.

These tensions provide an analytical lens for the remainder of the survey. Section 4 examines how different representation forms expose different kinds of dependencies. Section 5 discusses why tool-use provenance must move beyond event logging toward influence tracking and runtime control. Section 6 analyzes why memory provenance requires source lineage, temporal validity, and downstream influence tracking. Section 7 then considers whether existing benchmarks evaluate these provenance capabilities directly or only through final outcomes.

Table 3: Representative agent systems mapped onto the taxonomy dimensions of Section 3 (trace source, granularity, timing, representation, primary trust function). "Substrate" marks systems that generate traces but do not themselves implement a trust function; "(eval)" marks benchmarks that probe a capability rather than enforce it. The mapping exposes that current systems each occupy a narrow region of the design space.

System	Trace source(s)	Granularity	Timing	Representation	Primary trust function
ReAct Yao et al. [2023]	Reasoning, tool, env	Step	Runtime	Reasoning–action log	Substrate
Reflexion Shinn et al. [2023]	Reasoning, memory	Step	Continuous	Episodic memory	Recovery
Generative Agents Park et al. [2023]	Memory, env	Step	Continuous	Memory stream	Substrate
Toolformer Schick et al. [2023]	Tool	Tool-call	Pre (learned)	—	Substrate
ToolLLM Qin et al. [2024]	Tool	Tool-call, param	Runtime	API-call path	Substrate
WebArena Zhou et al. [2024]	Env, tool	Step	Post-hoc	Action/state log	Debugging (eval)
τ-bench Yao et al. [2025]	Tool, env	Tool-call	Post-hoc	State log	Verification (eval)
AutoGen Wu et al. [2024a]	Multi-agent, tool	Step	Runtime	Message log	Substrate
ALCE Gao et al. [2023]	Retrieval	Claim	Post-hoc	Citation links	Attribution
FActScore Min et al. [2023]	Retrieval	Claim (atomic)	Post-hoc	Claim–evidence	Verification
RAGAS Es et al. [2024]	Retrieval	Answer	Post-hoc	—	Verification
RAGChecker Ru et al. [2024]	Retrieval	Claim	Post-hoc	Claim–evidence	Debugging
SourceCheckup Wu et al. [2024b]	Retrieval	Claim	Post-hoc	Source–support	Attribution
ToolEmu Ruan et al. [2024]	Tool	Tool-call	Runtime (emul.)	Action log	Safety (eval)
InjecAgent Zhan et al. [2024]	Tool, env	Tool-call	Runtime	—	Safety (eval)
AgentDojo Debenedetti et al. [2024]	Tool, env	Tool-call	Runtime	—	Safety (eval)
CaMeL Debenedetti et al. [2025]	Tool	Parameter	Runtime	Control/data flow	Safety
FIDES Costa et al. [2025]	Tool, memory	Parameter	Runtime	IFC labels	Safety
NeuroTaint Cai et al. [2026]	Tool, memory	Param/token	Runtime	Taint labels	Safety
Agent-Sentry Sequeira et al. [2026]	Tool	Parameter	Runtime	Provenance graph	Safety, Audit
AgentSpec Wang et al. [2025a]	Tool, env	Tool-call	Pre/Runtime	Policy rules	Safety
AgentBound Bühler et al. [2025]	Tool	Tool-call	Pre/Runtime	Permissions	Safety
MemGPT Packer et al. [2023]	Memory	Step	Continuous	Tiered memory	Substrate
A-MEM Xu et al. [2026]	Memory	Step	Continuous	Linked memory	Substrate
AgentOps / AgentTrace Dong et al. [2024], AlSayyad et al. [2026]	All sources	Step	Post-hoc/Cont.	Structured log	Audit, Debugging
TRAIL Deshpande et al. [2025]	All sources	Step	Post-hoc	Annotated trace	Debugging
MAST Cemri et al. [2026]	Multi-agent	Step	Post-hoc	Failure taxonomy	Debugging

4 Provenance Representation and Evidence Attribution

The previous section identifies what should be traced; this section turns to how those objects and relations are represented. We view provenance representation as a family of structures that turn raw traces into inspectable, queryable, and auditable records: structured logs record execution events, execution graphs organize dependencies, evidence and claim-support graphs capture grounding relations, and specialized dependency graphs represent tool, memory, and multi-agent influence. These forms bridge the taxonomy above and the later discussions of tool-use safety, memory provenance, evaluation, and governance.

4.1 From Structured Logs to Execution Graphs

A natural starting point for agent provenance is the structured execution log. Instead of storing only free-form transcripts, structured logs record typed events such as user requests, model outputs, retrieval calls, tool invocations, tool outputs, memory operations, environment observations, inter-agent messages, and final responses. W3C PROV-DM provides the general provenance vocabulary of entities, activities, agents, and relations [World Wide Web Consortium, 2013], while PROV-AGENT adapts this view to agentic workflows by modeling prompts, responses, decisions, tool interactions, and downstream workflow context [Souza et al., 2025]. AgentOps and AgentTrace operationalize this idea for LLM agents: the former frames observability as tracking agent artifacts and their relationships across the agent lifecycle, while the latter proposes structured operational, cognitive, and contextual logs for agent execution [Dong et al., 2024, AlSayyad et al., 2026]. In short, agent traces should capture intermediate artifacts, execution context, tool interactions, and state changes, not just input–output pairs.

However, structured logs mainly provide chronological observability, whereas provenance-aware analysis requires dependency structure. A log can record that retrieval, tool calls, memory operations, and generation occurred in sequence, but it may not show which evidence, memory item, or tool output influenced a later claim or action. Execution graphs address this gap by representing instructions, evidence, tool calls, memory operations, claims, and actions as connected provenance objects. TRAIL illustrates this direction by using annotated traces to localize failures to specific steps, components, or interactions [Deshpande et al., 2025]. Thus, structured logs make behavior observable, while execution graphs make dependencies explicit. Recent failure-attribution work extends this idea from observed execution graphs to counterfactual and contrastive traces. AgenTracer constructs failed multi-agent trajectories through counterfactual replay and programmed fault injection, enabling failures to be attributed to responsible agents and

Table 4: Main provenance representation forms for LLM agents.

Form	What it captures	Main role
Structured logs	Typed execution events such as requests, retrieval calls, tool calls, memory operations, observations, and outputs.	Makes agent behavior observable and reconstructable.
Execution graphs	Dependencies among instructions, evidence, tool calls, memory items, claims, and actions.	Makes execution influence explicit for debugging and audit.
Evidence graphs	Links between evidence units, citations, atomic claims, and generated answers.	Distinguishes citation presence, relevance, support, contradiction, and omission.
Static schemas	Object types, relation types, trust labels, and provenance fields.	Provides a reusable vocabulary for trace representation and comparison.
Runtime provenance	Concrete source-to-sink, tool, memory, and action dependencies during execution.	Supports monitoring, enforcement, recovery, and information-flow analysis.

steps [Zhang et al., 2025a]. Aegis creates positive–negative trajectory pairs by injecting context-aware errors into successful multi-agent executions and annotating faulty agents and error modes [Kong et al., 2025]. These approaches show that provenance representations can support not only post-hoc inspection, but also supervised diagnosis, contrastive learning, and actionable repair feedback.

4.2 Evidence Graphs and Claim-Support Attribution

Evidence attribution requires a more semantic representation than execution ordering. The key question is not only whether evidence was retrieved or cited, but whether a generated claim is actually supported by the evidence it depends on. Prior work motivates this view at different levels of granularity. ALCE treats citations as explicit links between generated answers and supporting sources [Gao et al., 2023], while FActScore decomposes long-form generations into atomic facts whose support can be checked against reliable sources [Min et al., 2023]. SourceCheckup further shows that a cited source may be topically relevant without supporting the specific generated claim [Wu et al., 2024b]. PaperTrail makes this structure explicit by decomposing both LLM-generated answers and source documents into claims and evidence units, then mapping supported claims, unsupported claims, and omitted evidence [Martin-Boyle et al., 2026]. Citation presence, source relevance, claim support, and evidence omission are thus distinct relations that provenance should represent separately.

This view motivates evidence and claim-support graphs that connect evidence units to the claims, decisions, or actions they support. Such graphs distinguish evidence availability from evidence use, making grounded generation not only a generation problem but also a provenance representation problem.

4.3 Static Schemas and Runtime Provenance

A final design dimension is the distinction between static provenance schemas and dynamic runtime provenance. A static schema defines the object and relation types that may appear in provenance records, such as evidence units, tool calls, memory items, claims, actions, and relations including SUPPORT, DEPEND-ON, CONTRADICT, UPDATE, and INVALIDATE. W3C PROV-DM provides a general foundation for this view through domain-independent concepts such as entities, activities, agents, and derivation relations [World Wide Web Consortium, 2013]. For LLM agents, this schema must be specialized to include agent-specific objects such as retrieved chunks, tool arguments, memory writes, inter-agent messages, generated claims, and external state changes.

Dynamic runtime provenance records the concrete objects and dependencies produced in a particular execution. AgentOps and AgentTrace emphasize runtime trace capture and observability, while Agent-Sentry, FIDES, and NeuroTaint show that runtime provenance can support enforcement, information-flow control, and source-to-sink influence analysis [Dong et al., 2024, AlSayyad et al., 2026, Sequeira et al., 2026, Costa et al., 2025, Cai et al., 2026]. The key trade-off is granularity: fine-grained provenance supports verification, debugging, recovery, and audit, but increases storage overhead, logging complexity, privacy exposure, and annotation burden; coarse-grained provenance is easier to collect, but may fail to identify which evidence supported a claim, which memory item caused an error, or which tool argument crossed a trust boundary.

Table 4 summarizes the main representation forms discussed in this section.

4.4 Synthesis: Representation Trade-offs

The representation forms reviewed in this section expose different levels of dependency in agent execution. Structured logs provide chronological observability: they record what happened and when, but they do not by themselves explain whether a retrieved passage supported a claim, whether a tool call was justified, or whether a memory item influenced a downstream decision. Evidence and claim-support graphs provide stronger semantic attribution, but they often remain centered on document-to-answer grounding and may miss tool arguments, memory updates, environment state changes, or inter-agent message flows. Execution graphs and runtime provenance mechanisms move closer to system-level accountability because they connect actions, tools, observations, and intermediate artifacts, but they require fine-grained instrumentation and can introduce storage overhead, privacy exposure, and annotation complexity.

This comparison suggests that no single representation is sufficient for all provenance functions. Logs are useful for observability and coarse audit; evidence graphs are useful for claim verification; execution graphs are useful for debugging and dependency analysis; and runtime provenance is useful for enforcement and recovery. The key challenge is therefore to align representation granularity with the target trust function. A low-risk RAG agent may only require claim-level evidence links, whereas a tool-using enterprise agent may require parameter-level provenance and runtime source tracking. A long-term personalized agent may further require memory lineage, versioning, and invalidation support. Provenance representation should therefore be treated as a layered design problem rather than a single logging format.

5 Execution Provenance in Tool-Using Agents

This section examines the *outward* axis of agent provenance identified in the reading guide: settings where the agent acts on the external world. It instantiates the taxonomy of Section 3 in the place where the DEPEND-ON, TRIGGER, and UPDATE relations and runtime timing become decisive, and where the chronological-logging-versus-semantic-provenance and static-schema-versus-runtime-enforcement tensions are sharpest. Tool use is a central reason why execution provenance is necessary for LLM agents. Once an agent can call external tools, APIs, databases, code interpreters, browsers, or enterprise systems, its behavior is no longer limited to generating text: a tool call may retrieve private data, modify external states, send messages, execute code, or trigger irreversible downstream actions. Therefore, trustworthy tool use requires tracing not only final outputs, but also why a tool was called, where its arguments came from, whether its output was reliable, and how the result influenced later decisions. This section reviews provenance mechanisms for tool-call and argument tracing, indirect prompt injection, information-flow control, access boundaries, runtime guardrails, and trace-based debugging.

5.1 Tool-Call, Argument, and Output Provenance

Tool-call provenance records the execution path from tool selection to downstream influence: which tool was selected, which arguments were passed, what output was returned, and how that output affected later reasoning or actions. Tool-augmented systems such as Toolformer and ToolLLM show that external tools can expand LLM capabilities by enabling API calls, calculation, search, and other external operations Schick et al. [2023], Qin et al. [2024]. From a provenance perspective, however, the key question is not only whether a tool call succeeds, but whether the call is justified, whether its arguments come from trusted sources, and whether its output is used appropriately.

A tool call contains several provenance-critical objects: the selected tool, its schema, argument values, execution result, and downstream consumers. Among these, argument provenance is especially important because unsafe behavior often arises from passing untrusted or incorrectly generated values into otherwise legitimate tools. For example, a calendar or email tool may be safe in general, but unsafe if recipient fields, event descriptions, or message contents are copied from an untrusted webpage containing hidden instructions. Agent-Sentry directly addresses this problem by tracking the sources of values that flow into tool arguments, showing why tool-use safety requires parameter-level provenance rather than only tool-level permission checks Sequeira et al. [2026].

Tool outputs also become provenance-bearing execution units. A returned value may support a claim, update memory, trigger another tool call, modify external state, or conflict with other evidence. In web and tool-agent-user benchmarks, successful execution often depends on interpreting tool outputs correctly, satisfying policy constraints, and applying state-changing actions in the right order Zhou et al. [2024], Yao et al. [2025]. Thus, tool-use provenance should connect upstream sources, tool arguments, tool outputs, and downstream actions into a single dependency chain, rather than merely logging that a tool was called.

5.2 Indirect Prompt Injection and Tool-Use Contamination

Tool-using agents are vulnerable to indirect prompt injection because they consume external content that may contain adversarial instructions. Unlike direct prompt injection, where the malicious instruction is placed in the user prompt, indirect injection embeds the instruction in webpages, documents, emails, database entries, tool outputs, or third-party API responses. This attack exploits a core weakness of LLM-integrated systems: the boundary between data and instructions is often ambiguous, so untrusted content may be interpreted as executable guidance rather than passive evidence Greshake et al. [2023], Liu et al. [2023].

Recent benchmarks show that this risk grows as agents interact with richer tool environments. InjecAgent and AgentDojo evaluate indirect prompt injection and adversarial tool interactions involving external content, private data, and realistic task constraints Zhan et al. [2024], Debenedetti et al. [2024]. ToolEmu studies high-risk tool execution in an LM-emulated sandbox, enabling unsafe behavior to be evaluated without real-world side effects Ruan et al. [2024]. OpenAgentSafety extends evaluation to real tools such as browsers, code execution environments, file systems, bash shells, and messaging platforms Vijayvargiya et al. [2026], while MCP-SafetyBench targets real MCP servers and multi-server workflows with server-, host-, and user-side attack surfaces Zong et al. [2025]. Together, these benchmarks show that malicious external content or unsafe tool contexts can lead agents to invoke unauthorized tools, leak private information, ignore user intent, or perform unintended state-changing actions.

From a provenance perspective, indirect prompt injection is an information-flow and influence-tracking problem. The key question is not only whether malicious content was observed, but whether it influenced a later tool call, argument value, memory update, or external action. A flat log may show that a webpage was retrieved and an email tool was later called, but it may not reveal whether the email recipient or body was derived from the webpage, the user instruction, or the model's own reasoning. Provenance relations such as DERIVE, DEPEND-ON, TRIGGER, and USE make this influence explicit. Thus, tool-use provenance should distinguish trusted from untrusted sources and prevent untrusted content from crossing into privileged execution channels.

5.3 Information Flow, Taint Tracking, and Provenance Enforcement

Recent defenses increasingly frame tool-use safety as an information-flow problem. Rather than relying only on prompting or output filtering, they track where information originates, how it propagates through the agent, and whether it is allowed to influence sensitive actions. This view extends classical information-flow control, where security policies regulate flows among subjects, objects, and security classes Denning [1976], Sabelfeld and Myers [2003]. In LLM agents, the analogous problem is to prevent low-integrity or untrusted external data from controlling high-impact tool calls, memory updates, or state-changing actions.

A central challenge is that LLMs may treat untrusted data as executable instruction. CaMeL addresses this by separating control flow from data flow and restricting how external data can influence agent behavior Debenedetti et al. [2025]. Instruction hierarchy and StruQ pursue a related principle by distinguishing privileged instructions from lower-priority user or third-party data through training or structured prompt/data channels Wallace et al. [2024], Chen et al. [2025]. FIDES further formalizes this idea as agent-level information-flow control, tracking confidentiality and integrity labels and enforcing policies during execution Costa et al. [2025].

Semantic taint tracking and execution provenance provide finer-grained enforcement over values, arguments, and dependencies. NeuroTaint propagates taint across neural and symbolic components, while Agent-Sentry tracks whether sensitive tool arguments are influenced by untrusted sources Cai et al. [2026], Sequeira et al. [2026]. The lesson is that unsafe behavior often arises from *influence*, not content: a webpage may be safe to summarize, but unsafe if its hidden instruction becomes an email recipient, database parameter, or authorization signal. Provenance-aware enforcement makes this distinction explicit by connecting sources, transformations, and actions.

5.4 Access Control and Execution Boundaries

Information-flow tracking must be paired with access control and execution boundaries. Even if an agent can reason about many tools, it should not be allowed to invoke all tools under all contexts. This follows the classical security principles of least privilege and complete mediation: sensitive actions should require explicit authorization, and access decisions should be checked at the point of use Saltzer and Schroeder [1975]. In tool-using agents, these principles translate into tool permissions, argument constraints, user confirmations, sandboxing, and policy-mediated execution.

Recent agent-safety systems operationalize these ideas at runtime through explicit constraints: AgentSpec enforces rules over agent actions and tool use, and AgentBound bounds the unsafe action space Wang et al. [2025a], Bühler et al. [2025]; both are evaluated against the risky-execution and prompt-injection settings established by ToolEmu and AgentDojo Ruan et al. [2024], Debenedetti et al. [2024]. The shared lesson is that tool-use safety requires more than

detecting harmful text: it requires controlling which tools can be used, under what context, with which arguments, and with what external effects.

Provenance makes access control context-sensitive. A read-only search tool may safely consume untrusted web content, but an email, database-update, or code-execution tool may require arguments to be derived from trusted user intent, verified tool outputs, or explicit approval. Agent-Sentry illustrates this connection by checking whether sensitive tool arguments are influenced by untrusted sources Sequeira et al. [2026]. Thus, access control is not separate from provenance; it uses provenance to decide whether a proposed action should be allowed, downgraded, sandboxed, confirmed, or blocked.

5.5 Runtime Guardrails and Pre-/Post-Execution Verification

Runtime guardrails use provenance information to check agent behavior before, during, or after execution. Pre-execution guardrails decide whether a proposed tool call should be allowed; runtime guardrails monitor information flow and state changes during execution; post-execution verification inspects traces, outputs, and side effects for debugging, audit, and recovery. General guardrail systems such as NeMo Guardrails and Llama Guard show how external safety mechanisms can be placed around LLM applications, but tool-using agents require guardrails that reason over execution state, tool arguments, and downstream effects, not only input-output text Rebedea et al. [2023], Inan et al. [2023].

For high-impact tools, pre-execution verification is especially important. A provenance-aware guardrail can check whether the tool is authorized, whether user intent is present, whether arguments are well-formed, whether sensitive fields come from trusted sources, and whether the action violates policy. AgentSpec formalizes this idea through runtime constraints over agent actions, while Agent-Sentry applies provenance tracking to determine whether sensitive tool arguments are influenced by untrusted sources Wang et al. [2025a], Sequeira et al. [2026]. ToolEmu complements these defenses by evaluating risky tool executions in an emulated sandbox before real-world deployment Ruan et al. [2024].

Post-execution verification supports debugging, auditing, and recovery. When an agent produces a wrong answer or unsafe action, traces can help localize whether the failure came from retrieval, tool selection, parameter generation, tool execution, tool-output interpretation, memory update, or downstream reasoning. Observability systems such as AgentOps and AgentTrace record structured execution traces for inspection, while trace-based debugging systems such as TRAIL and LADYBUG use traces to localize failures and support repair Dong et al. [2024], AlSayyad et al. [2026], Deshpande et al. [2025], Rorseth et al. [2025].

Thus, effective guardrails require both semantic and procedural provenance. Semantic provenance checks whether evidence supports or contradicts a claim, while procedural provenance checks whether the execution path followed authorized steps and whether actions depended on trusted inputs. Together, they allow systems to detect unsupported claims, unsafe tool calls, contaminated arguments, invalid state updates, and cascading execution failures.

Table 5 summarizes representative systems according to their main focus, provenance object, enforcement timing, and role in tool-use provenance.

5.6 Summary

Tool-using agents make execution provenance necessary because tool calls can consume untrusted information, expose private data, modify external state, and trigger downstream consequences. The core provenance questions are where tool arguments come from, whether tool outputs are trustworthy, how external content influences actions, and whether execution remains within authorized boundaries. Recent work on prompt-injection benchmarks, information-flow control, taint tracking, execution bounding, runtime guardrails, observability, and trace-based debugging shows a clear shift from output-only safety toward provenance-aware execution control. This shift is central to trustworthy LLM agents: safe tool use requires not only deciding what the agent says, but also tracing and governing what the agent does.

Synthesis: from tool logging to influence control. The tool-use literature reveals a shift from event recording toward runtime influence control, but current methods still provide only partial accountability. Tool-level traces can show which API was called, which arguments were used, and what output was returned, but they do not necessarily show whether the call was necessary, justified, or consistent with the user's goal. Parameter-level provenance can identify whether untrusted values flow into tool arguments, but it may not determine whether the agent's interpretation of the tool output is semantically valid. Information-flow and taint-based methods can constrain unsafe influence, but they remain difficult to apply when information is transformed through summarization, paraphrasing, memory consolidation, or multi-step reasoning. Runtime guardrails can block unsafe actions, but they often operate under incomplete traces and may not support recovery after an unsafe or unsupported step has already affected memory or external state.

Table 5: Summary of representative work on tool-use execution provenance. The *Core mechanism* column states how each line of work operates technically, complementing the tracked objects and the provenance role it serves.

Line of Work	Representative Systems	Tracked Objects	Core mechanism (how)	Provenance Role
Tool-use learning	Toolformer Schick et al. [2023]; ToolLLM Qin et al. [2024]	Tool calls, API arguments, tool outputs	Learn when and how to call tools from (self-)supervised tool-use trajectories	Tool-call trace construction
Prompt-injection benchmarks	InjecAgent Zhan et al. [2024]; AgentDojo Debenedetti et al. [2024]; ToolEmu Ruan et al. [2024]	External content, injected instructions, private data	Embed adversarial instructions in external content and measure unsafe actions in (emulated) tool execution	Contamination and risk evaluation
Instruction–data separation	Instruction Hierarchy Wallace et al. [2024]; StruQ Chen et al. [2025]; CaMeL Debenedetti et al. [2025]	Privileged instructions, untrusted data, control/data flow	Mark privileged instructions vs. untrusted data and separate control flow from data flow via training or structured channels	Data–instruction boundary control
Information-flow tracking	FIDES Costa et al. [2025]; NeuroTaint Cai et al. [2026]	Security labels, tainted values, influence paths	Attach confidentiality/integrity (or taint) labels and propagate them source-to-sink, including through semantic transformations	Source-to-sink enforcement
Argument provenance	Agent-Sentry Sequeira et al. [2026]	Tool arguments, sources, trust labels	Build an execution-provenance graph and check whether sensitive tool arguments derive from untrusted sources	Sensitive-argument verification
Execution boundaries	AgentSpec Wang et al. [2025a]; AgentBound Bühler et al. [2025]; MCP-SafetyBench Zong et al. [2025]	Agent actions, permissions, policy constraints, and tool interfaces	Specify, enforce, or evaluate constraints over agent actions and tool access	Runtime action constraints and boundary-aware provenance
Guardrails	NeMo Guardrails Rebedea et al. [2023]; Llama Guard Inan et al. [2023]	Inputs, outputs, dialogue states, safety policies	External rule/classifier layer that filters inputs, outputs, and dialogue states against safety policies	Application-level safety filtering
Trace-based debugging	AgentOps Dong et al. [2024]; AgentTrace AlSayyad et al. [2026]; TRAIL Deshpande et al. [2025]; LADYBUG Rorseth et al. [2025]	Structured traces, artifacts, failure locations	Record structured execution traces and localize failures post hoc for audit and repair	Audit, localization, and recovery

These limitations show that tool-use provenance cannot stop at logging tool calls or tracking parameter sources. For trustworthy agents, tool provenance must connect low-level execution records with higher-level questions of intent, justification, policy compliance, downstream influence, and recoverability. This creates a need for provenance mechanisms that can jointly represent tool-call lineage, semantic evidence support, source trust, state changes, and recovery dependencies across the full agent workflow.

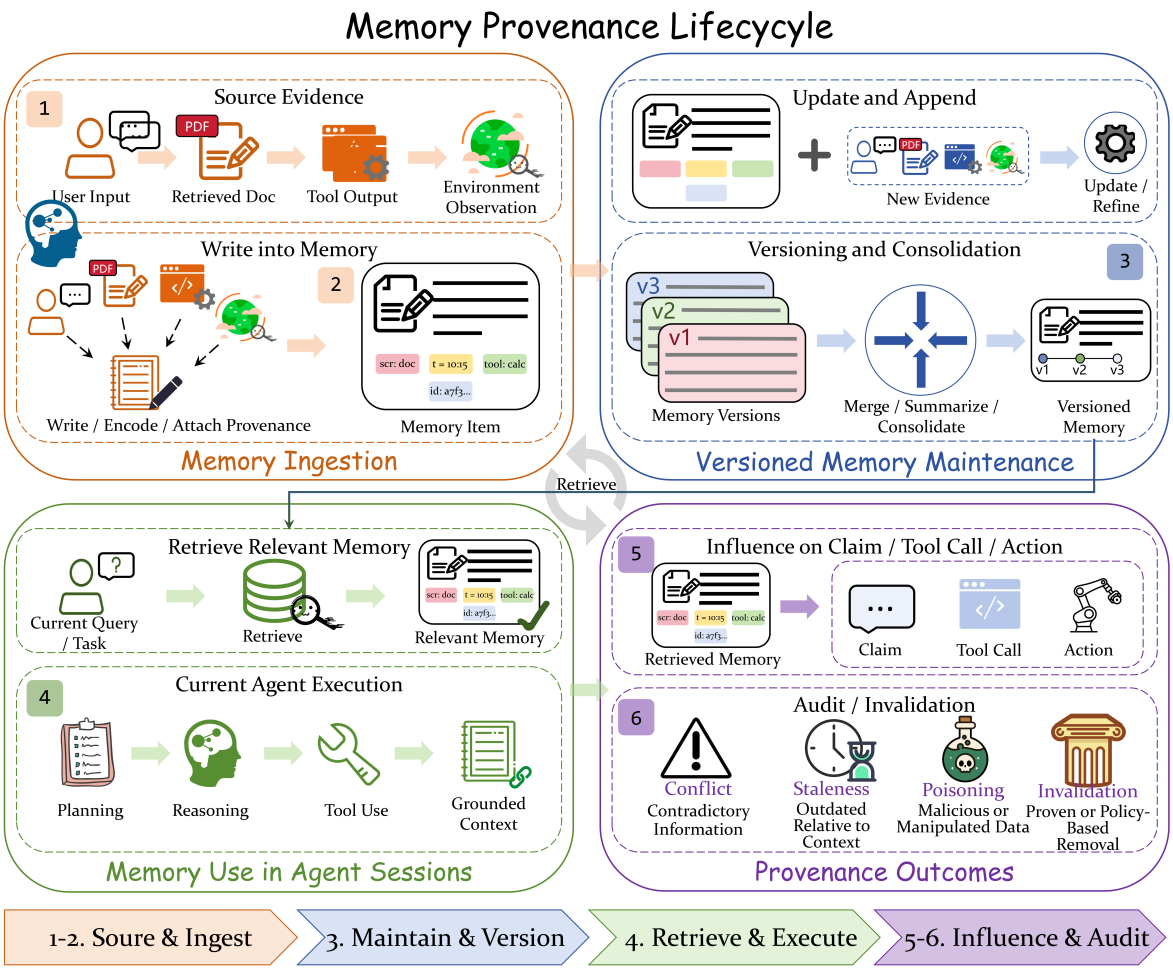


Figure 4: Memory as provenance-bearing evidence across agent sessions. Memory items are written from source evidence such as user input, retrieved documents, tool outputs, and environment observations; maintained through updates, versioning, and consolidation; retrieved into later agent executions; used to influence claims, tool calls, and actions; and audited for conflicts, staleness, poisoning, and invalidation.

6 Memory as Provenance-Bearing Evidence

This section examines the *inward* axis that complements tool use: where evidence is not acted upon externally but retained, transformed, and reused over time. It instantiates the taxonomy of Section 3 in the place where the DERIVE, CONTRADICT, and INVALIDATE relations and continuous timing become decisive, and where the trace-completeness-versus-deployability tension reappears as a question of what to persist across sessions. Memory enables LLM agents to maintain information across turns, tasks, sessions, and environments. It allows agents to recall prior observations, user preferences, feedback, tool results, and past decisions, thereby supporting long-term interaction and adaptive behavior. From the perspective of evidence tracing and execution provenance, however, memory is not merely a storage module. A memory item may later become evidence for claims, plans, tool calls, and final answers. Its origin, transformation, retrieval context, temporal validity, conflicts, and downstream influence therefore become part of the agent's provenance chain.

This section does not aim to provide another comprehensive survey of agent memory architectures. Existing surveys have already reviewed memory mechanisms, evaluation protocols, and graph-based memory systems for LLM agents [Zhang et al., 2025b, Du, 2026, Yang et al., 2026b]. Instead, we examine memory from a provenance perspective. We treat memory items as evidence-bearing objects whose sources, revisions, retrievals, and uses can affect later agent behavior. This view connects memory to the broader execution trace: a memory item may be written from an observation, derived from a tool output, updated through reflection, retrieved to support a claim, or invalidated by new evidence.

Unlike conventional memory diagrams that emphasize storage and retrieval, Figure 4 highlights memory as a provenance-bearing evidence object. It emphasizes not only how memory is written and retrieved, but also how it is updated, versioned, used in later executions, and audited for validity and risk.

6.1 Memory Writes, Source Attribution, and Lineage

Memory provenance begins when a memory item is written. A memory write may originate from user input, retrieved documents, environment observations, tool outputs, feedback, reflections, or messages from other agents. Existing agent memory systems illustrate the centrality of such write operations. Generative Agents store observations in a memory stream and use them for reflection and planning [Park et al., 2023]. Reflexion writes verbal feedback into episodic memory to improve future behavior after failures [Shinn et al., 2023]. MemoryBank maintains long-term memory through continuous updates from user interactions [Zhong et al., 2024]. MemGPT frames memory management as virtual context control, where information is moved between memory tiers through explicit read and write operations [Packer et al., 2023]. A-MEM further organizes memory dynamically by linking new memory items with existing ones in an interconnected structure [Xu et al., 2026].

From a provenance perspective, each memory write should carry source and lineage metadata, including its source type, timestamp, authoring agent, supporting evidence, transformation operation, confidence, and update history. This follows the broader provenance principle that generated artifacts should be linked to the entities, activities, and agents that produced them [World Wide Web Consortium, 2013, Souza et al., 2025], and is consistent with agent tracing frameworks that record execution artifacts and their operational context [AlSayyad et al., 2026, Sequeira et al., 2026]. Such lineage is especially important because memory writes often transform information: a document may be summarized into a memory item, multiple observations may be merged into a user profile, or a failed action may be abstracted into a reflection [Park et al., 2023, Shinn et al., 2023, Xu et al., 2026]. Without explicit lineage, these derived memories may later be reused as reliable context even when their origins, assumptions, or supporting evidence are uncertain. Memory provenance should therefore distinguish raw observations, extracted facts, inferred memories, and revised memories.

6.2 Memory Retrieval, Temporal Validity, and Downstream Influence

Memory retrieval determines which stored information enters the current execution context. Retrieved memories may influence task decomposition, answer generation, tool selection, argument construction, user modeling, and future memory updates. Long-term memory systems commonly retrieve prior memories to support future interactions: MemoryBank retrieves past memories for personalization [Zhong et al., 2024], MemGPT manages retrieval across memory tiers [Packer et al., 2023], LongMem studies long-term memory mechanisms for language models [Wang et al., 2023], Mem0 focuses on extracting and retrieving salient information across multi-session dialogues [Chhikara et al., 2025], and A-MEM improves retrieval through dynamic memory linking [Xu et al., 2026].

However, retrieval also introduces provenance risks. A retrieved memory may be partially relevant, outdated, overly general, derived from a previous hallucination, poisoned by adversarial interaction, or no longer valid in the current environment. Such risks are especially important because memory poisoning and injection attacks show that persistent memory can influence later responses and actions across sessions [Chen et al., 2024, Dong et al., 2025, Tian et al., 2026, Pulipaka et al., 2026]. Temporal validity is therefore a key part of memory provenance: a memory item should record not only its content, but also when it was created or updated, what evidence supported it, whether that evidence remains valid, and whether later observations have contradicted or superseded it.

Future provenance-aware retrieval should track not only which memories are retrieved, but also why they are retrieved and how they are used. Existing memory systems motivate tracking retrieved memory items and retrieval contexts [Packer et al., 2023, Chhikara et al., 2025, Xu et al., 2026], while provenance and observability frameworks motivate recording execution artifacts, sources, and downstream dependencies [World Wide Web Consortium, 2013, Dong et al., 2024, AlSayyad et al., 2026, Souza et al., 2025]. A provenance-aware memory trace should therefore record the triggering query or context, the retrieved memory items, their original sources, relevance signals, validity status, and downstream links to claims, tool calls, actions, final answers, or later memory updates. Such tracing would allow systems to determine whether a final output was influenced by a particular memory and whether that influence was appropriate. It would also support selective invalidation: when a memory is found to be stale, unsupported, private, or contaminated, the system could identify affected claims, actions, and derived memories rather than treating the execution history as opaque [Wang et al., 2025b, Wei et al., 2025, Sequeira et al., 2026].

6.3 Conflicting, Contradictory, and Poisoned Memory

Memory provenance must also account for conflict, contradiction, and contamination: stored memories may support incompatible claims, become invalidated by later evidence, or encode malicious content that is later reused as trustworthy

context. Recent security work makes this risk explicit. AgentPoison shows that poisoned long-term memory or RAG knowledge bases can serve as backdoor substrates for LLM agents [Chen et al., 2024]. Memory-injection attacks further demonstrate that malicious records can be induced through ordinary interactions without direct access to the memory store [Dong et al., 2025]. Environment-injected and sleeper memory poisoning studies show that untrusted observations from webpages, documents, or repositories can silently enter memory, persist across sessions, and later steer agent behavior [Zou et al., 2026, Pulipaka et al., 2026]. These findings suggest that memory quality cannot be evaluated only by recall or personalization performance; it also depends on source trust, write-path legitimacy, conflict status, contamination risk, cross-session persistence, and downstream activation.

This problem also connects memory provenance to tool-use safety and information-flow control. Fides tracks confidentiality and integrity labels to control unsafe information flows in AI agents [Costa et al., 2025], while NeuroTaint emphasizes that unsafe influence can persist through semantic transformations and memory rather than appearing only as exact text reuse [Cai et al., 2026]. Indirect prompt-injection benchmarks such as InjecAgent and AgentDojo further show that untrusted external content can manipulate tool-using agents [Zhan et al., 2024, Debenedetti et al., 2024]. When such content is written into persistent memory, the risk extends beyond a single execution and becomes a long-term provenance problem.

6.4 Memory-Grounded Verification and Long-Term Audit

Memory-grounded verification extends evidence tracing from retrieved documents and current tool outputs to persistent memory. When a claim or tool action relies on memory, the system should link that memory item to the observation, document, tool call, user statement, or reflection that created it, as well as to later revisions, conflicts, invalidations, and downstream uses. Claim-evidence interfaces such as PaperTrail motivate this granularity by decomposing LLM answers and source documents into discrete claims and evidence units [Martin-Boyle et al., 2026], while agent observability systems such as AgentOps and AgentTrace provide foundations for recording agent executions and structured operational, cognitive, and contextual logs [Dong et al., 2024, AlSayyad et al., 2026]. However, memory audit requires more than recording memory operations inside a single run. It requires persistent lineage across memory writes, retrieval events, updates, conflicts, invalidations, and later uses. Provenance-based runtime systems such as Agent-Sentry are especially relevant because they use execution provenance graphs to record the origins of tool-call argument values and support source-aware enforcement [Sequeira et al., 2026]. Long-term memory requires an analogous structure for persistent evidence.

Despite progress in long-term memory architectures and systems such as MemGPT, A-MEM, and Mem0 [Packer et al., 2023, Xu et al., 2026, Chhikara et al., 2025], memory provenance remains underdeveloped. Existing systems often optimize for recall, personalization, efficiency, or long-term coherence, but provide limited support for tracing memory origins, temporal validity, conflicts, contamination, privacy exposure, and downstream influence. This gap is critical because persistent memory itself can become an attack surface. AgentPoison shows that long-term memory or RAG knowledge bases can be poisoned to induce malicious downstream behavior [Chen et al., 2024]; InjecMEM shows that targeted memory injection can steer later responses through ordinary agent interactions [Tian et al., 2026]; and sleeper memory poisoning demonstrates that adversarial external context can implant fabricated memories that remain dormant and influence future conversations or actions [Pulipaka et al., 2026]. Memory privacy studies further show that sensitive information stored in agent memory can be exposed through adversarial interactions [Wang et al., 2025b]. Defense work such as A-MemGuard suggests that agent memory should support self-checking and self-correction rather than only input-time filtering [Wei et al., 2025]. Future memory systems therefore need provenance-aware mechanisms for source attribution, versioning, conflict detection, taint tracking, privacy-aware access control, stale-memory invalidation, and evidence-grounded verification, shifting memory from a capability mechanism to an auditable basis for accountable agent behavior.

Synthesis: memory usefulness versus memory accountability. Memory provenance exposes a different tension from tool-use provenance. Memory systems are usually optimized for usefulness, continuity, and personalization, whereas provenance-aware memory requires source validity, temporal validity, conflict tracking, and downstream accountability. A compressed or consolidated memory may improve retrieval efficiency, but it can obscure the evidence from which the memory was derived. A persistent memory may improve long-term personalization, but it can also preserve stale, private, poisoned, or weakly supported information across sessions. Therefore, memory provenance should not only record memory reads and writes, but also maintain source lineage, update history, validity status, conflict relations, and downstream influence links. Without these structures, long-term memory becomes an opaque evidence source rather than an auditable component of agent execution.

Table 6: Agent memory through a provenance lens. Existing memory systems support long-term recall, memory updates, and adaptive behavior, but provenance-specific capabilities such as source attribution, conflict handling, staleness detection, contamination tracking, and evidence-aware verification remain underdeveloped. Entries follow a common rubric: *Yes* = the capability is explicitly designed and supported; *Partial/Limited* = supported only indirectly or for a subset of cases; *No* = not addressed; *Attack-focused* = the system targets the corresponding failure mode adversarially rather than as a defense.

Paper / System	Memory Type	Write Provenance	Retrieval Provenance	Update / Evolution	Conflict Handling	Staleness Handling	Evidence-Aware Verification
Generative Agents [Park et al., 2023]	Memory stream / reflection	Partial	Partial	Reflection-based	No	No	No
Reflexion [Shinn et al., 2023]	Episodic memory / feedback	Partial	Partial	Feedback-driven	No	No	Partial
MemoryBank [Zhong et al., 2024]	Long-term interaction memory	Partial	Yes	Continuous update	Limited	Limited	No
MemGPT [Packer et al., 2023]	Hierarchical virtual memory	Yes	Yes	Memory-tier management	Limited	Limited	No
LongMem [Wang et al., 2023]	Long-term memory for LMs	Partial	Yes	Cache and update	No	Partial	No
Mem0 [Chhikara et al., 2025]	Production long-term memory	Yes	Yes	Extraction and consolidation	Limited	Partial	Partial
A-MEM [Xu et al., 2026]	Agentic interconnected memory	Yes	Yes	Dynamic linking and evolution	Limited	Partial	No
AgentPoison / Memory Injection [Chen et al., 2024, Dong et al., 2025]	Poisoned or injected memory	Attack-focused	Attack-focused	Cross-session persistence	No	No	No
Fides / NeuroTaint [Costa et al., 2025, Cai et al., 2026]	Information-flow and taint tracking	Yes	Yes	Tracks influence	Yes	Partial	Yes
AgentOps / AgentTrace [Dong et al., 2024, AlSayyad et al., 2026]	Execution logs and traces	Partial	Partial	Trace-level history	No	No	Partial
Agent-Sentry [Sequeira et al., 2026]	Execution provenance graph	Yes	Yes	Runtime provenance	Partial	Partial	Yes

7 Benchmarks, Datasets, and Metrics

Evaluation is central to evidence tracing and execution provenance. Beyond final-answer correctness, provenance-aware agent evaluation must assess evidence support, tool-call justification, unsafe influence, trace completeness, failure localization, and recovery. Existing benchmarks provide partial views across RAG attribution, citation evaluation, agent execution, tool-use safety, memory, and multi-agent analysis, but few evaluate complete provenance across evidence, tools, memory, traces, safety, and recovery. This section summarizes representative benchmark families, metrics, and limitations.

To provide a high-level view of these gaps, Figure 5 summarizes the coverage of major benchmark families across provenance-related evaluation capabilities.

The heatmap shows that existing benchmark families tend to specialize in one or two dimensions, such as evidence grounding, tool use, memory, multi-agent interaction, or safety attacks. This motivates the more detailed discussion of benchmark families below.

7.1 Benchmark Families for Evidence Tracing

RAG and attribution benchmarks provide the closest foundation for evidence tracing because they evaluate whether generated outputs are grounded in retrieved or cited evidence [Gao et al., 2023, Es et al., 2024, Saad-Falcon et al., 2024, Min et al., 2023, Niu et al., 2024, Ru et al., 2024, Wu et al., 2024b]. These benchmarks generally focus on the relationship between a final answer and its supporting sources, rather than the full execution trajectory of an agent. Nevertheless, they provide important metrics and datasets for evaluating evidence recall, citation quality, faithfulness, and claim-level support.

At the answer level, ALCE treats citations as explicit links between answer spans and external sources and checks whether those citations support the generated content, while RAGAS and ARES score reference-free faithfulness, context relevance, and answer relevance using lightweight model judges [Gao et al., 2023, Es et al., 2024, Saad-Falcon

Benchmark Family	Evidence Labels	Tool Calls	Memory	Multi-agent	Safety Attacks	Provenance Relations	Recovery
RAG benchmarks	S	L	L	L	P	P	L
Tool-use benchmarks	L	S	L	P	P	P	L
Memory benchmarks	P	L	S	L	P	P	P
Multi-agent benchmarks	L	P	L	S	P	P	L
Trace-debugging benchmarks	P	P	P	P	L	P	P
Provenance/security benchmarks	P	S	P	P	S	S	P

Legend: **S** Strong coverage **P** Partial coverage **L** Limited or absent coverage. *Rating criteria:* a capability is rated **S** when it is a primary evaluation target with dedicated labels or metrics, **P** when it is supported only indirectly or for a subset of cases, and **L** when it is largely absent.

Key gap. Existing benchmarks cover important but isolated aspects of agent provenance. No benchmark family provides strong end-to-end coverage across evidence labels, tool calls, memory, multi-agent communication, safety attacks, provenance relations, and recovery.

Figure 5: Benchmark coverage heatmap across provenance-related evaluation capabilities. The heatmap summarizes benchmark families at a high level rather than listing individual datasets. It highlights that existing benchmarks provide partial coverage of isolated components, while full-stack evaluation of evidence tracing and execution provenance remains underdeveloped.

et al., 2024]. These frameworks share one assumption—that an answer should be supported by retrieved evidence—but evaluate it only at the answer or context level, not per claim.

Finer-grained benchmarks move toward claim-level provenance, differing mainly in the unit they score: retrieval-versus-generation error separation (RAGChecker), word-level hallucination annotation (RAGTruth), atomic-fact support (FActScore), source-supportiveness of cited references (SourceCheckup), and supported/refuted/unverifiable claim classification (FEVER) [Ru et al., 2024, Niu et al., 2024, Min et al., 2023, Wu et al., 2024b, Thorne et al., 2018]. The trend is a shift from answer-level to claim-level grounding; SourceCheckup sharpens the point by showing that citation presence does not imply evidence support.

These benchmarks are important for evidence tracing, but they remain limited from an agent-provenance perspective. Their primary evaluation targets are answer-source grounding, citation support, faithfulness, hallucination annotation, or claim-level factual support [Gao et al., 2023, Es et al., 2024, Saad-Falcon et al., 2024, Min et al., 2023, Niu et al., 2024, Ru et al., 2024, Wu et al., 2024b]. As a result, they provide only partial coverage of agent provenance: they help evaluate whether claims are supported by evidence, but they do not directly evaluate how tool calls, memory operations, environment observations, intermediate decisions, or multi-agent communication shape the final answer or action. Thus, they should be viewed as foundations for evidence attribution rather than complete benchmarks for execution provenance.

7.2 Benchmarks for Agent Execution Traces and Tool-Use Safety

Agent benchmarks extend evaluation from static answer generation to interactive, multi-step execution, exposing trajectories, tool calls, environment states, and intermediate decisions. They differ mainly in setting and in what they score: multi-environment capability (AgentBench), long-horizon web interaction (WebArena), API selection and execution paths (ToolBench/ToolLLM), and tool-agent-user workflows that grade policy following and final database state rather than only the final response (τ-bench) [Liu et al., 2024, Zhou et al., 2024, Qin et al., 2024, Yao et al., 2025]. The common trend is a move from scoring the answer toward scoring the process and its end state.

Trace-oriented benchmarks connect execution provenance to failure diagnosis. AgentTrace provides structured operational, cognitive, and contextual logs for agent observability and downstream audit [AlSayyad et al., 2026], while TRAIL uses human-annotated traces to evaluate whether systems can localize failures to specific steps, components, or interactions [Deshpande et al., 2025]. Recent multi-agent work moves further from trace inspection to explicit failure attribution: AgenTracer constructs TracerTraj through counterfactual replay and fault injection to identify responsible

agents and steps [Zhang et al., 2025a], and Aegis generates positive–negative trajectory pairs with faulty-agent and error-mode labels [Kong et al., 2025]. Together, these works show that provenance can be evaluated not only by trace completeness, but also by its ability to support failure localization and attribution.

Tool-use safety benchmarks examine whether agents remain safe when tools introduce external risks. ToolEmu evaluates high-risk tool use in an LM-emulated sandbox [Ruan et al., 2024]; InjecAgent and AgentDojo study indirect prompt injection and adversarial tool interactions in tool-integrated settings [Zhan et al., 2024, Debenedetti et al., 2024]; and OpenAgentSafety extends evaluation to real tools and multi-turn, multi-user tasks across multiple risk categories [Vijayvargiya et al., 2026]. MCP-SafetyBench further studies safety risks in real MCP servers and multi-server workflows [Zong et al., 2025]. These benchmarks matter for provenance because unsafe behavior often depends on hidden flows from untrusted sources into tool arguments, execution boundaries, or state-changing actions.

Recent work evaluates more explicit execution-provenance and information-flow mechanisms. Agent-Sentry introduces execution provenance graphs to bound LLM-agent behavior and detect out-of-bounds tool calls [Sequeira et al., 2026]. NeuroTaint and TaintBench evaluate semantic taint and source-to-sink influence in LLM agents, emphasizing that unsafe influence may be transformed through summarization, inference, or memory rather than copied verbatim [Cai et al., 2026]. Fides applies information-flow control to secure AI agents by tracking confidentiality and integrity labels [Costa et al., 2025]. AgentSpec evaluates runtime policy enforcement through customizable constraints [Wang et al., 2025a]. AgentBound extends this line of work to execution boundaries and access control in MCP-style agent-tool ecosystems [Bühler et al., 2025]. Together, these benchmarks and systems shift evaluation from "can the agent use tools?" to "can the agent use tools safely, with traceable and enforceable provenance?"

7.3 Memory and Multi-Agent Provenance Evaluation

Memory and multi-agent settings introduce additional provenance challenges because information may persist across sessions, be updated over time, or propagate across agents. Existing memory benchmarks and systems mainly evaluate recall, personalization, long-term consistency, memory updating, retrieval, or multi-session task performance [Zhong et al., 2024, Packer et al., 2023, Xu et al., 2026, Chhikara et al., 2025, Maharana et al., 2024, Kim et al., 2026]. MemoryBank evaluates long-term memory recall, continuous updating, and personalization [Zhong et al., 2024]; MemGPT makes memory read and write operations explicit through hierarchical memory management [Packer et al., 2023]; A-MEM organizes agent memory through dynamically evolving interconnected memory structures [Xu et al., 2026]; Mem0 evaluates memory extraction, consolidation, and retrieval in multi-session dialogue settings [Chhikara et al., 2025]; LOCOMO evaluates long-term conversational memory [Maharana et al., 2024]; and MemoryArena evaluates memory use in interdependent multi-session agentic tasks [Kim et al., 2026]. Existing memory and graph-memory surveys further summarize progress in memory mechanisms, evaluation protocols, and graph-based memory systems [Zhang et al., 2025b, Du, 2026, Yang et al., 2026b].

From a provenance perspective, these benchmarks provide useful but incomplete proxies for memory provenance. They often test whether an agent remembers useful information, but rarely evaluate whether a memory item is traceable to its source, temporally valid, consistent with newer evidence, protected from contamination, or responsible for a downstream action. Dedicated memory-provenance evaluation should therefore assess memory source attribution, update history, conflict resolution, stale-memory invalidation, contamination detection, and downstream influence, which remain underdeveloped in current memory evaluation protocols [Maharana et al., 2024, Kim et al., 2026, Zhang et al., 2025b, Du, 2026].

Multi-agent systems introduce another layer of provenance complexity because evidence, assumptions, messages, and errors can propagate across agent boundaries. AutoGen and CAMEL provide representative frameworks for multi-agent communication, role assignment, and collaborative task solving [Wu et al., 2024a, Li et al., 2023]. Evaluating provenance in such settings requires more than measuring final task success: it requires identifying which agent contributed which evidence, which communication step introduced an error, and how responsibility is distributed across agents. MAST analyzes failures from specification errors, inter-agent misalignment, and weak task verification [Cemri et al., 2026], while TRAIL evaluates issue localization from agent traces [Deshpande et al., 2025]. AgenTracer and Aegis further construct labeled multi-agent trajectories for identifying faulty agents, responsible steps, and error modes [Zhang et al., 2025a, Kong et al., 2025]. Together, these works suggest that multi-agent provenance benchmarks should evaluate evidence propagation, communication-level error introduction, cross-agent dependency, and responsibility attribution, rather than only final coordination success.

7.4 Metrics for Evidence Tracing and Execution Provenance

Existing benchmarks use diverse metrics for retrieval grounding, citation quality, tool use, safety, memory, and agent execution [Gao et al., 2023, Min et al., 2023, Ru et al., 2024, Wu et al., 2024b, Ruan et al., 2024, Debenedetti et al.,

Evaluation Aspect	Core Question	Representative Metrics	Status	Related Basis
Evidence attribution	Are the agent's claims supported by reliable evidence?	Evidence recall, citation precision, source supportiveness, faithfulness, claim support accuracy.	Established	RAG and attribution evaluation [Gao et al., 2023, Min et al., 2023, Niu et al., 2024, Ru et al., 2024, Wu et al., 2024b].
Execution provenance	Is the agent's execution process completely and correctly traceable?	Trace completeness, provenance accuracy, dependency coverage, temporal consistency.	Proposed	Provenance standards and agent observability [World Wide Web Consortium, 2013, Dong et al., 2024, AlSayyad et al., 2026, Souza et al., 2025].
Safety and robustness	Can the system detect unsafe or untrusted influence during execution?	Unsafe influence detection, attack success rate, policy violation rate, intervention precision.	Established	Tool-use safety, prompt injection, and information-flow control [Ruan et al., 2024, Zhan et al., 2024, Debenedetti et al., 2024, 2025, Costa et al., 2025, Zong et al., 2025].
Debugging and recovery	Can traces help identify and repair failures?	Failure localization accuracy, diagnosis correctness, auditability, recovery success.	Partly established	Trace-based debugging, multi-agent failure analysis, and failure-attribution datasets [Deshpande et al., 2025, Cemri et al., 2026, AlSayyad et al., 2026, Zhang et al., 2025a, Kong et al., 2025].

Table 7: Representative metric dimensions for evaluating evidence tracing and execution provenance in LLM agents. The **Status** column flags maturity: *Established* metrics are operationalized in existing RAG, attribution, tool-use, or safety benchmarks; *Partly established* metrics exist for some sub-tasks (e.g., failure localization in TRAIL/MAST) but not for recovery; *Proposed* metrics (trace completeness, provenance accuracy, dependency coverage, temporal consistency) are desiderata that are not yet operationalized with agreed definitions, and we mark them as such to avoid conflating them with mature measures.

2024, Maharana et al., 2024, Deshpande et al., 2025, Cemri et al., 2026]. From the perspective of evidence tracing and execution provenance, these metrics can be reorganized into four evaluation aspects: evidence attribution, execution provenance, safety and robustness, and debugging and recovery. Table 7 summarizes representative metric dimensions and their related foundations. Some metrics are already used in RAG, attribution, tool-use, and agent benchmarks, while others are natural extensions for provenance-aware evaluation.

The table highlights that provenance-aware evaluation should go beyond final-answer correctness and component-level task success. Building on prior work in attribution evaluation, provenance modeling, agent observability, tool-use safety, and trace-based debugging [Gao et al., 2023, Min et al., 2023, Ru et al., 2024, World Wide Web Consortium, 2013, Dong et al., 2024, AlSayyad et al., 2026, Souza et al., 2025, Ruan et al., 2024, Debenedetti et al., 2024, Deshpande et al., 2025, Cemri et al., 2026], we organize these metrics by their evaluation function. Evidence attribution metrics assess claim grounding; execution-provenance metrics assess traceability; safety and robustness metrics assess unsafe influence and policy violations; and debugging and recovery metrics assess whether traces support failure localization, audit, and repair. Together, these dimensions provide a broader evaluation view for agents whose outputs depend on retrieval, tools, memory, communication, and external state changes.

7.5 Limitations of Current Evaluation Protocols

Existing benchmarks provide useful foundations for evaluating retrieval grounding, tool use, memory, web interaction, safety, and multi-agent behavior, but they remain insufficient for full evidence tracing and execution provenance. RAG and attribution benchmarks mainly focus on answer-source grounding or citation quality [Gao et al., 2023, Ru et al., 2024, Wu et al., 2024b]. Tool-use and agent benchmarks emphasize task success, unsafe tool behavior, prompt injection, environment interaction, real-world tool safety, or MCP-specific attacks [Liu et al., 2024, Zhou et al., 2024, Ruan et al., 2024, Debenedetti et al., 2024, Yao et al., 2025, Vijayvargiya et al., 2026, Zong et al., 2025]. Memory and multi-agent benchmarks evaluate recall, personalization, coordination, failure modes, or error attribution [Maharana et al., 2024, Kim et al., 2026, Deshpande et al., 2025, Cemri et al., 2026, Zhang et al., 2025a, Kong et al., 2025]. Although recent work has improved realistic tool-safety evaluation and multi-agent failure attribution, existing benchmarks still rarely evaluate the complete provenance chain from evidence and tool outputs to memory updates, intermediate claims, final answers, external state changes, and recovery actions.

A key limitation is the lack of unified execution traces and provenance-relation annotations. Trace-oriented benchmarks and observability frameworks have made useful progress, but they still differ in what artifacts they record and how provenance is represented [Dong et al., 2024, AlSayyad et al., 2026, Souza et al., 2025, Deshpande et al., 2025, Cemri et al., 2026]. Few benchmarks jointly capture user requests, retrieved evidence, tool calls and outputs, memory reads and writes, inter-agent messages, intermediate claims, final responses, and state changes. Even fewer annotate relations such as SUPPORT, CONTRADICT, DEPEND-ON, UPDATE, and INVALIDATE, despite the availability of general provenance concepts in standards such as W3C PROV-DM [World Wide Web Consortium, 2013]. As a result, current evaluation often measures correctness, faithfulness, or attack success indirectly, rather than directly assessing trace completeness, provenance accuracy, claim support, memory validity, or cross-agent responsibility.

Recovery and governance are also under-evaluated. Existing benchmarks commonly test whether an agent succeeds, fails, hallucinates, or takes an unsafe action, but rarely evaluate whether provenance can help the system repair execution, such as by invalidating stale memory, quarantining contaminated evidence, retrying a tool call, requesting human approval, or rolling back an unsafe state change [Ruan et al., 2024, Debenedetti et al., 2024, Deshpande et al., 2025, Cemri et al., 2026]. Realistic evaluation is further complicated by long-horizon and enterprise traces involving private tools, changing data, user-specific memory, organizational policies, credentials, and confidential intermediate states [Maharana et al., 2024, Kim et al., 2026, Dong et al., 2024, AlSayyad et al., 2026]. Future provenance benchmarks should therefore evaluate not only whether agents succeed, but also whether their behavior is traceable, auditable, safe, privacy-aware, and recoverable [Costa et al., 2025].

Table 8 summarizes representative benchmarks and datasets according to their task type, trace availability, evidence labels, tool calls, memory, multi-agent setting, safety attacks, and main metrics.

8 Open Problems and Future Directions

Despite rapid progress in RAG attribution, agent observability, tool-use safety, memory systems, graph-based representation, debugging, and multi-agent evaluation, evidence tracing and execution provenance for LLM agents remain fragmented. Existing systems can record execution artifacts, but logging alone does not explain how those artifacts jointly shape an agent's final answer or action. This section summarizes open problems toward provenance-aware agents that support verification, auditing, attribution, safety, recovery, and governance.

8.1 Toward Unified and Interoperable Trace Schemas

A major challenge is the lack of unified trace schemas for LLM agents. Existing agent frameworks, observability tools, and benchmarks record different artifacts using different formats. Some systems log prompts and model responses; others log tool calls, retrieved documents, memory reads and writes, environment observations, execution metadata, or error events [Dong et al., 2024, AlSayyad et al., 2026, Souza et al., 2025, Deshpande et al., 2025, Cemri et al., 2026]. Few provide interoperable schemas that jointly represent reasoning steps, retrieval, tool calls, tool outputs, memory operations, inter-agent messages, intermediate claims, final answers, and external state changes.

Traditional provenance and tracing standards provide useful starting points, although they were not designed specifically for LLM agents. W3C PROV-DM defines general provenance concepts such as entities, activities, agents, usage, generation, and derivation [World Wide Web Consortium, 2013]. OpenTelemetry and OpenLineage provide infrastructure-level abstractions for distributed traces and data workflow lineage [OpenTelemetry, 2026, OpenLineage Contributors, 2026]. Agent-specific frameworks such as AgentOps and AgentTrace move closer to LLM-agent observability by recording agent artifacts and structured operational, cognitive, and contextual logs [Dong et al., 2024, AlSayyad et al., 2026]. PROV-AGENT further adapts provenance modeling to agentic workflows [Souza et al., 2025]. However, a unified

Table 8: Benchmarks for evidence tracing and execution provenance. Existing benchmarks cover different parts of the provenance problem, including citation quality, RAG faithfulness, agent execution traces, tool-use safety, memory evaluation, and multi-agent failure analysis. However, few benchmarks jointly provide evidence labels, tool calls, memory operations, multi-agent communication, safety perturbations, and provenance-relation annotations.

Benchmark	Task Type	Trace Available?	Evidence Labels?	Tool Calls?	Memory?	Multi-Agent?	Safety Attacks?	Main Metrics
ALCE [Gao et al., 2023]	Citation generation	Partial	Yes	No	No	No	No	Citation quality, correctness, fluency
RAGAS [Es et al., 2024]	RAG evaluation	No	Partial	No	No	No	No	Faithfulness, context relevance, answer relevance
ARES [Saad-Falcon et al., 2024]	RAG evaluation	No	Partial	No	No	No	No	Context relevance, answer faithfulness, answer relevance
RAGChecker [Ru et al., 2024]	Fine-grained RAG diagnosis	Partial	Partial	No	No	No	No	Retrieval and generation diagnosis
RAGTruth [Niu et al., 2024]	RAG hallucination annotation	Partial	Yes	No	No	No	No	Word-level hallucination labels
FActScore [Min et al., 2023]	Atomic factuality	No	Yes	No	No	No	No	Atomic fact support
SourceCheckup [Wu et al., 2024b]	Source supportiveness	No	Yes	No	No	No	No	Source relevance and supportiveness
AgentBench [Liu et al., 2024]	Agent capability evaluation	Partial	No	Partial	Task-dependent	No	No	Task success, environment-specific scores
WebArena [Zhou et al., 2024]	Web-agent execution	Yes	No	Yes	No	No	No	Task success, web interaction outcome
ToolBench / ToolLLM [Qin et al., 2024]	API tool use	Yes	No	Yes	No	No	No	Tool selection, argument correctness, execution success
τ-bench [Yao et al., 2025]	Tool-agent-user interaction	Yes	No	Yes	State-based	No	Partial	Task success, policy following, final state
TRAIL [Deshpande et al., 2025]	Trace debugging	Yes	Partial	Partial	Partial	Partial	No	Issue localization, error type diagnosis
MAST [Cemri et al., 2026]	Multi-agent failure analysis	Yes	No	Task-dependent	Task-dependent	Yes	No	Failure taxonomy, error attribution
ToolEmu [Ruan et al., 2024]	Tool-use risk evaluation	Yes	No	Yes	No	No	Yes	Risk detection, unsafe action rate
InjecAgent [Zhan et al., 2024]	Indirect prompt injection	Yes	No	Yes	No	No	Yes	Attack success rate, defense performance
AgentDojo [Debenedetti et al., 2024]	Prompt injection attacks and defenses	Yes	No	Yes	No	No	Yes	Attack success, utility, defense robustness
Agent-Sentry Bench [Sequeira et al., 2026]	Out-of-bounds tool execution	Yes	Partial	Yes	No	No	Yes	Out-of-bounds detection, false positives
NeuroTaint / TaintBench [Cai et al., 2026]	Semantic taint tracking	Yes	Partial	Yes	Yes	Partial	Yes	Unsafe influence detection, taint propagation
AgentBound [Bühler et al., 2025]	MCP execution boundary	Yes	No	Yes	No	No	Yes	Access-control violations, policy enforcement
LOCOMO [Maharana et al., 2024]	Long-term conversational memory	Partial	Partial	No	Yes	No	No	Long-range QA, event summarization, memory recall
MemoryArena [Kim et al., 2026]	Multi-session agent memory	Yes	Partial	Task-dependent	Yes	No	No	Multi-session task success, memory use

schema is still needed to connect operational execution events with semantic evidence relations across retrieval, tool use, memory, multi-agent communication, and state changes.

Future LLM-agent provenance schemas should connect operational execution events with semantic evidence relations. Existing standards and frameworks provide useful building blocks [World Wide Web Consortium, 2013, OpenTelemetry, 2026, OpenLineage Contributors, 2026, Souza et al., 2025], but they do not yet fully cover provenance across retrieval, tool use, memory, multi-agent communication, and external state changes. A unified schema should represent key entities, such as claims, evidence units, tool observations, memory items, environment states, and agent messages, together with relations such as SUPPORT, CONTRADICT, DEPEND-ON, UPDATE, and INVALIDATE. Such schemas would make agent traces easier to compare, replay, annotate, audit, and exchange across frameworks.

8.2 From Coarse Attribution to Claim-Level and Semantic Provenance

Current attribution methods often operate at coarse granularity, such as answer-level citation, context-level faithfulness, or step-level logging [Gao et al., 2023, Ru et al., 2024, Wu et al., 2024b, Schreieder et al., 2025]. However, LLM-agent outputs frequently contain multiple claims, intermediate assumptions, and action-relevant inferences. A final response may be partially correct: one claim may be supported by retrieved evidence, another may be inferred beyond the source, and another may be contradicted by a tool output or memory item. This motivates claim-level provenance, which builds on fine-grained factuality and attribution evaluation but extends it to agentic execution settings [Min et al., 2023, Niu et al., 2024].

Existing work provides useful foundations. ALCE studies citation generation and citation quality [Gao et al., 2023]; FActScore evaluates atomic fact support [Min et al., 2023]; RAGTruth provides fine-grained hallucination annotations [Niu et al., 2024]; RAGChecker diagnoses RAG systems at a finer granularity [Ru et al., 2024]; and SourceCheckup

evaluates whether cited sources are actually relevant and supportive [Wu et al., 2024b]. Broader surveys of attribution, citation, and quotation further show that evidence-based text generation still lacks unified definitions and evaluation protocols [Schreieder et al., 2025].

For agents, the problem is harder because provenance must track semantic influence across execution. LLMs may paraphrase, summarize, combine, infer, and transform information before using it in a claim or action. Evidence may pass through retrieval, tool calls, memory updates, reflection, or another agent's intermediate judgment. String-level matching is therefore insufficient. NeuroTaint highlights this issue by emphasizing semantic taint and causal influence in LLM agents [Cai et al., 2026]. Future work should develop methods for tracing semantic dependencies, including cases where evidence is transformed, compressed, aggregated, or indirectly used across multiple steps.

8.3 Memory and Multi-Agent Provenance

Memory provenance remains underexplored. Existing memory systems allow agents to store, retrieve, update, and reuse information across sessions, but they often provide limited support for tracing where memory items came from, whether they remain valid, whether they conflict with newer evidence, and how they influence later claims or actions. Memory surveys and graph-memory surveys have reviewed memory architectures, evaluation methods, and relational memory structures [Zhang et al., 2025b, Du, 2026, Yang et al., 2026b], but memory as provenance-bearing evidence remains a distinct open problem.

Future memory systems should record source attribution, write time, update history, retrieval context, downstream usage, conflict status, and invalidation events. Memory should not be treated as an unqualified context reservoir. Instead, memory items should carry provenance metadata indicating whether they were derived from user input, retrieved evidence, tool outputs, environment observations, reflections, or messages from other agents. This is especially important when memory affects tool calls, personalization, or long-term decision-making. Security work such as AgentPoison and memory-injection attacks shows that poisoned or maliciously induced memory can influence future agent behavior without modifying model weights [Chen et al., 2024, Dong et al., 2025].

Multi-agent provenance introduces additional complexity because agents may communicate, delegate tasks, verify one another's outputs, and jointly produce final decisions. Frameworks such as AutoGen and CAMEL demonstrate these distributed workflows [Wu et al., 2024a, Li et al., 2023], while MAST shows that failures can arise from specification errors, inter-agent misalignment, weak verification, and coordination breakdowns [Cemri et al., 2026]. AgenTracer and Aegis further show that responsible-agent and responsible-step attribution can be operationalized through labeled, counterfactual, or contrastive multi-agent trajectories [Zhang et al., 2025a, Kong et al., 2025]. Future provenance models should therefore track how claims, evidence, messages, and errors propagate across agents: which agent introduced a claim, which agent verified or reused it, which message carried it, which step induced failure, and where responsibility lies. The remaining challenge is to integrate responsibility attribution with claim-level evidence provenance, memory lineage, and recovery actions, rather than treating failure attribution as a separate diagnostic task.

8.4 Provenance-Aware Runtime Safety and Recovery

Runtime safety for LLM agents requires more than output filtering. In tool-using agents, dangerous behavior may arise when untrusted content influences tool arguments, when private information flows into public outputs, when previously stored context affects a privileged action, or when an agent invokes a tool outside its intended boundary. Guardrails that inspect only the final answer or the current tool call may miss these risks. Provenance-aware guardrails should instead reason over the source chain behind an action, including which evidence, memory items, tool outputs, and external observations influenced the action [Zhan et al., 2024, Debenedetti et al., 2024, 2025, Costa et al., 2025, Sequeira et al., 2026, Bühler et al., 2025].

Recent work points toward this direction. ToolEmu evaluates high-risk tool-use behavior [Ruan et al., 2024]; InjecAgent and AgentDojo evaluate indirect prompt injection in tool-integrated agents [Zhan et al., 2024, Debenedetti et al., 2024]; CaMeL separates trusted control flow from untrusted data flow [Debenedetti et al., 2025]; Fides applies information-flow control to AI agents [Costa et al., 2025]; NeuroTaint traces semantic influence [Cai et al., 2026]; AgentSpec provides runtime constraint enforcement [Wang et al., 2025a]; Agent-Sentry uses execution provenance graphs to bound agent behavior [Sequeira et al., 2026]; and AgentBound studies execution boundaries and access control in MCP-style agent-tool ecosystems [Bühler et al., 2025].

However, runtime provenance remains difficult. Guardrails must balance safety and utility, avoid excessive false positives, and operate under incomplete or noisy traces. Moreover, most current systems focus on detection or blocking, while recovery remains underdeveloped. Future provenance-aware agents should not only detect unsafe or unsupported execution, but also repair it. For example, an agent should be able to invalidate stale memory, quarantine contaminated

evidence, retry a failed tool call with corrected parameters, ask for human approval, roll back an unsafe state change, or replay execution from a safe checkpoint. This requires provenance representations that support intervention and recovery, not just explanation, building on provenance modeling, runtime enforcement, information-flow control, and execution-provenance mechanisms [World Wide Web Consortium, 2013, Souza et al., 2025, Costa et al., 2025, Wang et al., 2025a, Sequeira et al., 2026].

8.5 Realistic Benchmarks, Privacy, and Governance

Current benchmarks still lack realistic full execution traces. RAG benchmarks provide evidence labels but usually omit tools and memory. Tool-use safety benchmarks provide attacks and tool calls but often lack claim-level evidence labels. Memory benchmarks evaluate recall or personalization but rarely evaluate memory lineage, conflict handling, or downstream influence. Multi-agent benchmarks analyze failures but often lack complete provenance annotations. Benchmarks such as AgentBench, WebArena, τ-bench, TRAIL, MAST, LOCOMO, and MemoryArena provide important foundations [Liu et al., 2024, Zhou et al., 2024, Yao et al., 2025, Deshpande et al., 2025, Cemri et al., 2026, Maharana et al., 2024, Kim et al., 2026], but no existing benchmark fully evaluates evidence tracing and execution provenance across retrieval, tool use, memory, multi-agent communication, safety attacks, and recovery.

Future benchmarks should move from isolated answer-, task-, or attack-level evaluation toward trace-level evaluation. Building on provenance standards and agent observability frameworks [World Wide Web Consortium, 2013, Dong et al., 2024, AlSayyad et al., 2026, Souza et al., 2025], they should capture complete execution records across retrieval, tool use, execution boundaries, memory, environment interaction, inter-agent communication, and state changes. These traces should not only record events, but also annotate the relations that make them accountable, including support, contradiction, dependency, triggering, update, invalidation, unsafe influence, faulty-agent responsibility, and recovery intervention. Recent work on trace-based failure analysis, multi-agent attribution, real-world tool safety, and protocol-level tool environments shows that these dimensions are each important, but they are still evaluated mostly in isolation [Deshpande et al., 2025, Cemri et al., 2026, Zhang et al., 2025a, Kong et al., 2025, Vijayvargiya et al., 2026, Zong et al., 2025]. Future benchmarks should integrate these directions with claim-level evidence support, memory validity, unsafe influence detection, auditability, and recovery.

Privacy and governance are equally important. Provenance traces may contain sensitive user data, confidential documents, credentials, tool outputs, API arguments, internal reasoning artifacts, and long-term memory items. Observability frameworks such as AgentOps and AgentTrace provide useful foundations for collecting traces [Dong et al., 2024, AlSayyad et al., 2026], and security mechanisms such as Fides highlight the need to track confidentiality and integrity constraints [Costa et al., 2025]. However, future systems must also support trace minimization, access control, anonymization, retention policies, encryption, and selective disclosure. Without such mechanisms, provenance infrastructure itself may become a source of privacy and compliance risk.

Overall, realistic provenance-aware benchmarks must be developed together with privacy and governance mechanisms, so that agent systems can be evaluated not only by task success, but also by traceability, auditability, safety, and recoverability.

9 Conclusion

LLM agents are moving beyond isolated text generation toward systems that retrieve evidence, plan intermediate steps, invoke tools, update memory, interact with environments, and coordinate with other agents. This shift makes final-answer correctness insufficient for trustworthiness. Reliable agent systems require not only accurate outputs, but also the ability to trace where answers come from, which evidence supports them, how tools and intermediate actions contribute to them, and where failures or unsafe influences enter the execution process.

In this survey, we position evidence tracing and execution provenance as a unified perspective for studying trustworthy LLM agents. We organize retrieval grounding, citation faithfulness, tool-use logging, memory auditing, agent evaluation, and security risks around a common question: how information, decisions, and actions can be traced across the full agent lifecycle. We review key provenance representations, including structured logs, execution graphs, evidence graphs, tool dependency graphs, memory provenance, and multi-agent communication traces, and discuss their roles in attribution, debugging, verification, safety enforcement, evaluation, and recovery.

Our review shows that the field has made important progress, but remains fragmented. Existing systems often record sources or tool calls, yet they rarely provide fine-grained claim-level support, semantic influence tracking, memory lineage, cross-agent provenance propagation, or robust tracing under adversarial conditions. Looking forward, evidence tracing and execution provenance should become a first-class infrastructure layer for reliable agent systems. We hope

this survey helps consolidate this emerging research space and motivates the development of traceable, auditable, and recoverable LLM agents.

References

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=vAElhFcKW6.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=dHng2O0Jjr.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=oKn9c6ytLx.

Shunyu Yao, Noah Shinn, Parth Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=roNSXZpUDN.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=GEcwtMk1uA.

Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=m1YYAQjO3w.

Sanidhya Vijayvargiya, Aditya Bharat Soni, Xuhui Zhou, Zora Zhiruo Wang, Nouha Dziri, Graham Neubig, and Maarten Sap. OpenAgentSafety: A comprehensive framework for evaluating real-world AI agent safety. In *International Conference on Learning Representations*, 2026.

Xuanjun Zong, Zhiqi Shen, Lei Wang, Yunshi Lan, and Chao Yang. MCP-SafetyBench: A benchmark for safety evaluation of large language models with real-world MCP servers. *arXiv preprint arXiv:2512.15163*, 2025.

Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. Trail: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025. URL https://arxiv.org/abs/2505.08638.

Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. Why do multi-agent llm systems fail? *Advances in Neural Information Processing Systems*, 38, 2026.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the LLM agentic systems? *arXiv preprint arXiv:2509.03312*, 2025a.

Fanqi Kong, Ruijie Zhang, Huaxiao Yin, Guibin Zhang, Xiaofei Zhang, Ziang Chen, Zhaowei Zhang, Xiaoyuan Zhang, Song-Chun Zhu, and Xue Feng. Aegis: Automated error generation and attribution for multi-agent systems. *arXiv preprint arXiv:2509.14295*, 2025.

World Wide Web Consortium. PROV-DM: The PROV data model. W3C Recommendation, 2013. URL https://www.w3.org/TR/prov-dm/.

OpenTelemetry. Opentelemetry specification: Traces. Online documentation, 2026. URL https://opentelemetry.io/docs/specs/otel/.

Liming Dong, Qinghua Lu, and Liming Zhu. Agentops: Enabling observability of llm agents. *arXiv preprint arXiv:2411.05285*, 2024. URL https://arxiv.org/abs/2411.05285.

Adam AlSayyad, Kelvin Yuxiang Huang, and Richik Pal. Agenttrace: A structured logging framework for agent system observability. *arXiv preprint arXiv:2602.10133*, 2026. URL https://arxiv.org/abs/2602.10133.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488, Singapore, 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.emnlp-main.398.

Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100. Association for Computational Linguistics, 2023. URL https://aclanthology.org/2023.emnlp-main.741/.

Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta, 2024. Association for Computational Linguistics. URL https://aclanthology.org/2024.eacl-demo.16/.

Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 338–354, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.naacl-long.20. URL https://aclanthology.org/2024.naacl-long.20/.

Dongyu Ru, Lin Qiu, Xiangkun Hu, Tianhang Zhang, Peng Shi, Shuaichen Chang, Cheng Jiayang, Cunxiang Wang, Shichao Sun, Huanyu Li, Zizhao Zhang, Binjie Wang, Jiarong Jiang, Tong He, Zhiguo Wang, Pengfei Liu, Yue Zhang, and Zheng Zhang. Ragchecker: A fine-grained framework for diagnosing retrieval-augmented generation. In *Advances in Neural Information Processing Systems*, 2024.

Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, Kashun Shum, Randy Zhong, Juntong Song, and Tong Zhang. Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 10862–10878. Association for Computational Linguistics, 2024. doi:10.18653/v1/2024.acl-long.585. URL http://papers.nips.cc/paper_files/paper/2024/hash/27245589131d17368cccdfa990cbf16e-Abstract-Datasets_and_Benchmarks_Track.html.

Christoph Bühler, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi. Securing ai agent execution. *arXiv preprint arXiv:2510.21236*, 2025. URL https://arxiv.org/abs/2510.21236.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2023. doi:10.1145/3586183.3606763.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024. doi:10.1609/aaai.v38i17.29946.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604, 2026.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024a.

Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in neural information processing systems*, 36: 51991–52008, 2023.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: A large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 809–819. Association for Computational Linguistics, 2018. doi:10.18653/v1/N18-1074.

Rohan Sequeira, Stavros Damianakis, Umar Iqbal, and Konstantinos Psounis. Agent-sentry: Bounding llm agents via execution provenance. *arXiv preprint arXiv:2603.22868*, 2026. URL https://arxiv.org/abs/2603.22868.

Kevin Wu, Eric Wu, Ally Cassasola, Angela Zhang, Kevin Wei, Teresa Nguyen, Sith Riantawan, Patricia Shi Riantawan, Daniel E. Ho, and James Zou. How well do llms cite relevant medical references? an evaluation framework and analyses. *arXiv preprint arXiv:2402.02008*, 2024b. URL https://arxiv.org/abs/2402.02008.

Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666*, 2025a. URL https://arxiv.org/abs/2503.18666.

Joel Rorseth, Parke Godfrey, Lukasz Golab, Divesh Srivastava, and Jarek Szlichta. Ladybug: An llm agent debugger for data-driven applications. In *Proceedings of the 28th International Conference on Extending Database Technology*, pages 1082–1085, 2025. URL https://openproceedings.org/2025/conf/edbt/paper-313.pdf.

Xixun Lin, Yucheng Ning, Jingwen Zhang, Yan Dong, Yilong Liu, Yongxuan Wu, Xiaohua Qi, Nan Sun, Yanmin Shang, Pengfei Cao, Lixin Zou, Xu Chen, Chuan Zhou, Jia Wu, Shirui Pan, Bin Wang, Yanan Cao, Kai Chen, Songlin Hu, and Li Guo. Llm-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions. *arXiv preprint arXiv:2509.18970*, 2025. URL https://arxiv.org/abs/2509.18970.

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025. URL https://arxiv.org/abs/2503.18813.

Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing ai agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025. URL https://arxiv.org/abs/2505.23643.

Yuandao Cai, Wensheng Tang, Cheng Wen, and Shengchao Qin. Ghost in the agent: Redefining information flow tracking for llm agents. *arXiv preprint arXiv:2604.23374*, 2026. URL https://arxiv.org/abs/2604.23374.

Tobias Schreieder, Tim Schopf, and Michael Färber. Attribution, citation, and quotation: A survey of evidence-based text generation with large language models. *arXiv preprint arXiv:2508.15396*, 2025. URL https://arxiv.org/abs/2508.15396.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, Findings of ACL, pages 10471–10506. Association for Computational Linguistics, 2024. URL https://doi.org/10.18653/v1/2024.findings-acl.624.

Victor Ojewale, Harini Suresh, and Suresh Venkatasubramanian. Audit trails for accountability in large language models. *arXiv preprint arXiv:2601.20727*, 2026.

Yi Nian, Aojie Yuan, Haiyue Zhang, Jiate Li, and Yue Zhao. Auditable agents. *arXiv preprint arXiv:2604.05485*, 2026.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pages 57734–57811, 2024.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*, 2025.

Edward Y. Chang and Longling Geng. SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning. *Proceedings of the VLDB Endowment*, 18(12):4874–4886, 2025. doi:10.14778/3750601.3750611.

Ke Yang, Panpan Li, Zonghan Wu, Kejin Xu, Huaxi Huang, and Xiaoshui Huang. DART: Semantic recoverability for structured tool agents. *arXiv preprint arXiv:2605.23311*, 2026a.

Yusheng Zheng, Yiwei Yang, Wei Zhang, and Andi Quinn. ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore. *arXiv preprint arXiv:2603.20625*, 2026.

Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira Da Silva. Prov-agent: Unified provenance for tracking ai agent interactions in agentic workflows. In *2025 IEEE International Conference on eScience (eScience)*, pages 467–473. IEEE, 2025.

Anna Martin-Boyle, Cara Leckey, Martha Brown, and Harmanpreet Kaur. Papertrail: A claim-evidence interface for grounding provenance in llm-based scholarly q&a. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pages 1–25, 2026.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90. Association for Computing Machinery, 2023. doi:10.1145/3605764.3623985. URL https://doi.org/10.1145/3605764.3623985.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023. URL https://arxiv.org/abs/2306.05499.

Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi:10.1145/360051.360056. URL https://doi.org/10.1145/360051.360056.

Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003. doi:10.1109/JSAC.2002.806121.

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL https://arxiv.org/abs/2404.13208.

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. {StruQ}: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400, 2025.

Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi:10.1109/PROC.1975.9939.

Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445, Singapore, December 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.emnlp-demo.40. URL https://aclanthology.org/2023.emnlp-demo.40/.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023. URL https://arxiv.org/abs/2312.06674.

Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *ACM Transactions on Information Systems*, 2025b. doi:10.1145/3748302.

Pengfei Du. Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026. URL https://arxiv.org/abs/2603.07670.

Chang Yang, Chuang Zhou, Yilin Xiao, Su Dong, Luyao Zhuang, Yujing Zhang, Zhu Wang, Zijin Hong, Zheng Yuan, Zhishang Xiang, Shengyuan Chen, Huachi Zhou, Qinggang Zhang, Ninghao Liu, Jinsong Su, Xinrun Wang, Yi Chang, and Xiao Huang. Graph-based agent memory: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2602.05665*, 2026b. URL https://arxiv.org/abs/2602.05665.

Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. Augmenting language models with long-term memory. In *Advances in Neural Information Processing Systems*, volume 36, pages 74530–74543, 2023.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html.

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=QINnsnppv8.

Hanling Tian, Zeyang Sha, Jingying Wang, Yuhang Liu, Zhehao Huang, and Xiaolin Huang. InjecMEM: Memory injection attack on LLM agent memory systems. OpenReview preprint, 2026. URL https://openreview.net/forum?id=QVX6hcJ2um. Submitted to ICLR 2026.

Sidharth Pulipaka, Stanislau Hlebik, Leonidas Raghav, Sahar Abdelnabi, Vyas Raina, Ivaxi Sheth, and Mario Fritz. Hidden in memory: Sleeper memory poisoning in LLM agents, 2026. URL https://arxiv.org/abs/2605.15338.

Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. Unveiling privacy risks in LLM agent memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25241–25260, Vienna, Austria, July 2025b. Association for Computational Linguistics. doi:10.18653/v1/2025.acl-long.1227. URL https://aclanthology.org/2025.acl-long.1227/.

Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. A-MemGuard: A proactive defense framework for LLM-based agent memory, 2025. URL https://arxiv.org/abs/2510.02373.

Wei Zou, Mingwen Dong, Miguel Romero Calvo, Shuaichen Chang, Jiang Guo, Dongkyu Lee, Xing Niu, Xiaofei Ma, Yanjun Qi, and Jiarong Jiang. Poison once, exploit forever: Environment-injected memory poisoning attacks on web agents. *arXiv preprint arXiv:2604.02623*, 2026. doi:10.48550/arXiv.2604.02623.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 13851–13870, 2024.

Hyunjin Kim et al. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026. URL https://arxiv.org/abs/2602.16313.

OpenLineage Contributors. OpenLineage Specification. https://github.com/OpenLineage/OpenLineage/blob/main/spec/OpenLineage.md, 2026. Accessed: 2026-05-25.